CONTENTS

TOPICS **PAGE**
TEMPLATE 1
TEMPLATE JAVA 1
BIT OPERATIONS 1
DATA STRUCTURES
SEGMENT TREE 2
PERSISTENT SEGMENT TREE 2
BINARY INDEXED (FENWICK) TREE 2
AVL TREE 2
LINK CUT TREE 3
TRIES 4
MO ALGORITHM 5
MISOF TREE 5
HEAVY LIGHT DECOMPOSITION 5
DYNAMIC PROGRAMMING
LIS (LONGEST INCREASING SUBSEQUENCE) 6
CYCLIC LCS (LONGEST COMMON SUBSEQUENCE) 6
LCIS (LONGEST COMMON INCREASING SUBSEQUENCE) 7
DP Buka String & Lipat String 7
MATRIX CHAIN MULTIPLICATION 8
MIN CUT POLYGON TRIAGULATION 8
KADANE 8
GRAPHS
TOPOLOGICAL SORT 8
K-th SHORTEST PATH 8
MANHATTAN SPANNING TREE 9
DIRECTED MST 9
BELLMAN FORD (Negative Cycle Detection) 10
DINIC ALGORITHM 10
LOWEST COMMON ANCESTOR (SEG TREE) 10
MIN COST MAX FLOW 11
MIN COST MAX FLOW (Multiple Edge) 12
MAXFLOW (AdJ List) 12
MIN CUT (Stoer Wagner) 12
EULERIAN TOUR CHECKING 13
MAX BIPARTITE MATCHING 13
MINIMUM VERTEX COVER 13
ARTICULATION POINT & BRIDGE 14
TARJAN ALGORITHM 14
STRING PROCESSING
KMP ALGORITHM 14
SUFFIX ARRAY 14
ENHANCED SUFFIX ARRAY 14
EDIT DISTANCE 15
K-th PERMUTATION 15
ETC
Soal TIMO 2010 15
MEET IN THE MIDDLE 16
HUFFMAN CODES 16
HUNGARIAN ALGORITHM 16
STABLE MARRIAGE 16
CSOD 17
MATHEMATICS
BIG INTEGER 17
GAUSS ELIMINATION 19
NUMBER THEORY 19
MATHEMATICS SEQUENCE FORMULA 20
PRIME NUMBER LIST 20
CATALAN & SUPER CATALAN 20
DIGIT COUNTING FACTORIAL 20
INT TO ROMAN 20
nCk 20
LINEAR DIOPHANTINE 20
CRT dan kawan2 20
EULER TOTIENT (PHI) FUNCTION 21
PRIME TEST 21
POLLARD RHO 22
MILLER RABIN 22
EXTENDED EUCLID 22
SUM OF DIVISORS 22
RANGE SIEVE 22
MÖBIUS FUNCTION 22
GEOMETRIC TRANSFORMATION 22
TRIGONOMETRIC EQUATIONS 22
GEOMETRY
POLYGON CENTROID AND AREA 23
LINE SEGMENT INTERSECTION POINT 23
LINE SEGMENT INTERSECTION 23
SQUARE COLLISION 23
CLOSEST PAIR 23
POLYGON CLASS 24
PT TO LINE & LINE TO SEGMENT DIST 24
POLYGON DIAMETER 25
UNION RECTANGLE INTERSECTION 25
GREAT CIRCLE DISTANCE 25
END OF TEAM NOTEBOOK 25

END OF CONTENTS

TEMPLATE

```
#pragma GCC optimize (“O3”)
int irand(int lo, int hi){ // random
 return (((double)rand())/(RAND_MAX+1.0))*(hi-lo+1)+lo;
}
string toString(ll x) { // ll to string
    stringstream ss;  ss << x;  return ss.str();
}
ll toNumber(string S) { // string to ll
    ll ret;  sscanf(S.c_str(),"%lld",&ret);  return ret;
}
```

TEMPLATE JAVA

```
import java.math.BigInteger;
import java.util.Scanner;
import java.util.ArrayList;

class Main{
  static ArrayList alist = new ArrayList<BigInteger>();
    static void init(){
        BigInteger x = new BigInteger("1656");
        BigInteger y = BigInteger.valueOf(123);
        x = x.add(new BigInteger(“4498”));
        // substract  multiply  divide  remainder
        compare -> (A.compareTo(B)<0) (A.equals(B))
        int x = A.intValue();
        long y = A.longValue();
        String s = A.toString();
    }
    public static void Main(String[] args){
        init();   int x;
        Scanner in = new Scanner(System.in);
        while(in.hasNext()){
            System.out.println("Print "+in.NextInt());
        }
    }
}
```

BIT OPERATIONS

```
intA[30]; //Bilangan 2^N
void prework() { //Precompute 2^N
    A[0]=1;   for (int i=1;i<=30;i++) A[i]=A[i-1]*2;
}
Int getbit(int num, int pos){ //Get bit ke POS dari NUM
    return (num&A[pos])!= 0;
}
void setbit(int& num, int pos, int biner) {
    //Set bit ke POS jadi BINER di NUM
    int temp = getBit(num,pos);
    if (temp!=biner) num+=((temp<biner)?A[pos]:-A[pos]);
}
int removeLSOne (int num) { //Buang bit 1 paling kanan
    return num&(num-1);
}
__builtin_popcount(x) ada brp bit yang nyala
__builtin_clz(x) get number of leading 0 in x
__builtin_ctz(x) get number of trailing 0 in x
(x&(1<<y)) sisain bit ke-y+1  T=(S&(-S))bit pertama
x&=~(1<<y) matiin bit ke-y+1  yg nyala dari kanan
```

SEGMENT TREE

```
//array size: 2N                data inside N..2N-1

void build(){  // build the tree
    REV(i,n-1,0) t[i] = t[i<<1] + t[i<<1|1];
}
void updPURQ(int pos, int val){
    for (t[pos+=n] = value; pos>1; pos>>=1)
        t[pos>>1] = t[pos] + t[pos^1];
}
int queryPURQ(int l, int r){
    int res = 0;
    for(l+=n,r+=n; l<r; l>>=1,r>>=1){
        if(l&1)res += t[l++];
        if(r&1)res += t[--r];
    }
    return res;
}
void updRUPQ(int l, int r, int val){
    for(l+=n,r+=n; l < r; l>>=1,r>>=1){
        if(l&1) t[l++] += val;
        if(r&1) t[--r] += val;
} }
int queryRUPQ(int pos){
    int res = 0;
    for(p += n; p > 0; p>>=1) res += t[p];
    return res;
}
void pushRUPQ(){  // inspect all elements: O(n)
    REP(i,1,n-1){
        t[i<<1] += t[i];
        t[i<<1|1] += t[i];
        t[i] = 0;
    }
}
h = height of the tree (log)
void propoRURQ(int pos, int val){ // propagate lazy
    t[pos] += val;
    if(pos < n) lazy[pos] += value;
}
void updParRURQ(int pos){ // update all parents
    while(pos > 1){
        pos >>= 1;
        t[pos] = max(t[pos<<1],t[pos<<1|1]) + lazy[pos];
    }
}
void pushRURQ(int p){ // propagates from ancestor
    REV(i,h,1){
        int pos = p>>i;
        if(lazy[pos]){
            propoRURQ(pos<<1  ,lazy[pos]);
            propoRURQ(pos<<1|1,lazy[pos]);
            lazy[pos] = 0;
        }
    }
}
void updRURQ(int l, int r, int val) {
    l += n; r += n; int l0 = l, r0 = r;
    for(; l < r; l>>=1,r>>=1){
        if(l&1) propoRURQ(l++,val);
        if(r&1) propoRURQ(--r,val);
    }
    updParRURQ(l0);
    updParRURQ(r0-1);
}
int queryRURQ(int l, int r) {
    l += n; r += n; pushRURQ(l); pushRURQ(r - 1);
    int res = -2e9;
    for(; l < r; l >>= 1, r >>= 1) {
        if(l&1) res = max(res,t[l++]);
        if(r&1) res = max(res,t[--r]);
    }
    return res;
}
```

PERSISTENT SEGMENT TREE

```
struct Tree{
    int dat,L,R;
    Tree(){ dat=0; L=R=-1; }
} seg[2400005];
int num = 0;
int dat[100005], temp[100005], headList[100005];
map<int,int> hashh;   int rehashh[100005];

void build(int idx, int l, int r){
    if(l == r) return;
    int mid = (l+r)>>1;
    build(seg[idx].L = ++num,l,mid);
    build(seg[idx].R = ++num,mid+1,r);
}
void upd(int idx, int l, int r, int key, int prev){
    if(l==r){ seg[idx].dat = seg[prev].dat+1; return; }
    int mid = (l+r)>>1; seg[idx] = seg[prev];
    if(key<=mid)
        upd(seg[idx].L=++num,l,mid,key,seg[prev].L);
    else upd(seg[idx].R=++num,mid+1,r,key,seg[prev].R);
    seg[idx].dat=seg[seg[idx].L].dat+seg[seg[idx].R].dat
}
int que(int idx, int prev, int l, int r, int kth){
    if(l == r) return l;
    int LF = seg[seg[idx].L].dat-seg[seg[prev].L].dat;
    int mid = (l+r)>>1;
    if(kth <= LF)
        return que(seg[idx].L,seg[prev].L,l,mid,kth);
    return que(seg[idx].R,seg[prev].R,mid+1,r,kth-LF);
}
int main(){
    FOR(i,100003)dat[i]=0,headList[i]=rehashh[i]=-1;
    hashh.clear();
    //input
    int n,q, idx=0;
    scanf("%d %d",&n,&q);
    FOR(i,n) scanf("%d",dat+i),temp[i]=dat[i];
    sort(temp,temp+n); temp[n]=temp[n-1]+1;
    FOR(i,n) if(temp[i]!=temp[i+1])
        hashh[temp[i]] = ++idx, rehashh[idx] = temp[i];
    //build
    build(headList[0] = ++num,1,idx);
    FOR(i,n){
        headList[i+1] = ++num;
        upd(num,1,idx,hashh[dat[i]],headList[i]);
    }
    //query
    while(q--){
        int l,r,kth;
        scanf("%d %d %d",&l,&r,&kth);
        r = headList[r];   l = headList[l-1];
        printf("%d\n",rehashh[que(r,l,1,idx,kth)]);
    }
    return 0;
}
```

BINARY INDEXED (FENWICK) TREE

```
int LSOne(int n){ return (n&-n); }
void upd(int* dat, int pos, int size, int value){
    REPP(i,pos,n,LSOne(i)) dat[i] += value; // i|=i+1
}
int que(int* dat, int pos){
    int res = 0; // b=(b&(b+1))-1
    REVV(i,pos,1,LSOne(i)) res += dat[i];
    return res;
}
```

Range Update

```
upd(BITree1, l, val);     upd(BITree2, l, val*(l-1));
upd(BITree1, r+1, -val);  upd(BITree2, r+1, -val*r);
```

Range [1..k] Sum

```
que(BITTree1, k) *k) - que(BITTree2, k);
```

AVL TREE

```
// AVL Tree
```

```
struct Tree {
    int key, ubound;
    int height,size;
    Tree *left,*right;
    Tree(int key) {
        this->key = key; left = right = NULL;
        this->ubound = 0;
        height = 0; size = 1;
    }
}*root;

inline int height(Tree *p) {
    return p==NULL ? -1 : p->height;
}
inline int size(Tree *p) {
    return p==NULL ? 0 : p->size;
}
int upper_bound( Tree *p, int x ) {
    if ( p == NULL ) return 0;
    if ( p->key <= x ) {
        return p->ubound = size(p->left) + 1 +
            upper_bound( p->right, x );
    }
    return upper_bound( p->left, x );
}
inline void rotateL(Tree *&p) {
    Tree *c = p->left;
    p->left = c->right;
    c->right = p;
    p->height = max(height(p->left),height(p->right))+1;
    c->height = max( height(c->left), p->height ) + 1;
    p->size = size(p->left) + size(p->right) + 1;
    c->size = size(c->left) + size(c->right) + 1;
    p = c;
}
inline void rotateR(Tree *&p) {
    Tree *c = p->right;
    p->right = c->left;
    c->left = p;
    p->height = max(height(p->left),height(p->right))+1;
    c->height = max( height(c->right), p->height ) + 1;
    p->size = size(p->left) + size(p->right) + 1;
    c->size = size(c->left) + size(c->right) + 1;
    p = c;
}
void insert(int x,Tree *&p) {
    if(p==NULL) p = new Tree(x);
    else if(x<=p->key) {
        insert(x,p->left);
        if(height(p->left)-height(p->right) >= 2) {
            if(x>=p->left->key) rotateR(p->left);
            rotateL(p);
        }
    }
    else if(x>p->key) {
        insert(x,p->right);
        if(height(p->right)-height(p->left) >= 2) {
            if(x<=p->right->key) rotateL(p->right);
            rotateR(p);
        }
    }
    p->height = max(height(p->left),height(p->right))+1;
    p->size = size(p->left) + size(p->right) + 1;
    p->ubound = upper_bound( p, p->key );
}
int kth(Tree *p,int x) {
    if(size(p->left)+1==x) return p->key;
    if(x < size(p->left)+1) return kth(p->left,x);
    return kth(p->right,x-size(p->left)-1);
}
void printAll( Tree *p ) {
    if ( p == NULL ) return;
    printAll( p->left );
    printf (" %d ", p->key );
    printAll( p->right );
}
inline Tree * findMin(Tree *p) {
    while(p->left!=0) p=p->left;
    return p;
}
void removeAll( Tree *p ) {
    if ( p==NULL ) return;
    removeAll( p->left );
    removeAll( p->right );
    delete p;
}
void remove(int x,Tree *&p) {
    if(p==0) return;
    if(x<p->key) remove(x,p->left);
    else if(x>p->key) remove(x,p->right);
    else if(p->left!=0 && p->right!=0) {
        p->key = findMin(p->right)->key;
        remove(p->key,p->right);
    }
    else {
        Tree *temp = p;
        p = (p->left!=0) ? p->left:p->right;
        delete temp;
    }
    if(p!=0) {
        p->size = size(p->left) + size(p->right) + 1;
      p->height=max(height(p->left),height(p->right))+1;
    }
}
```

LINK CUT TREE

```
/*
LinkCut tree(n);   // new link-cut tree with n nodes
tree.link(x, y);   // link x and y
tree.cut(x);       // cut x
tree.root(x);      // root of tree containing x
tree.depth(x);     // depth of x in its tree
tree.lca(x, y);    // lowest common ancestor of x and y
*/
struct Node{
    int sz, label; /* size, label */
    Node *p, *pp, *l, *r; /* parent, path-parent, left,
right pointers */
    Node() { p = pp = l = r = 0; }
};
void update(Node *x){
    x->sz = 1;
    if(x->l) x->sz += x->l->sz;
    if(x->r) x->sz += x->r->sz;
}
void rotr(Node *x){
    Node *y, *z;   y = x->p, z = y->p;
    if((y->l = x->r)) y->l->p = y;
    x->r = y, y->p = x;
    if((x->p = z)){
        if(y == z->l) z->l = x;
        else z->r = x;
    }
    x->pp = y->pp;   y->pp = 0;   update(y);
}
void rotl(Node *x){
    Node *y, *z;   y = x->p, z = y->p;
    if((y->r = x->l)) y->r->p = y;
    x->l = y, y->p = x;
    if((x->p = z)){
        if(y == z->l) z->l = x;
        else z->r = x;
    }
    x->pp = y->pp;   y->pp = 0;   update(y);
}
void splay(Node *x){
    Node *y, *z;
    while(x->p){
        y = x->p;
```

```
        if(y->p == 0)}{
            if(x == y->l) rotr(x);
            else rotl(x);
        }
        else{
            z = y->p;
            if(y == z->l){
                if(x == y->l) rotr(y), rotr(x);
                else rotl(x), rotr(x);
            }
            else{
                if(x == y->r) rotl(y), rotl(x);
                else rotr(x), rotl(x);
            }
        }
    }
    update(x);
}
Node *access(Node *x){
    splay(x);
    if(x->r){
        x->r->pp = x;  x->r->p = 0;
        x->r = 0;      update(x);
    }
    Node *last = x;
    while(x->pp){
        Node *y = x->pp;   last = y;  splay(y);
        if(y->r) y->r->pp = y, y->r->p = 0;
        y->r = x;  x->p = y;  x->pp = 0;
        update(y); splay(x);
    }
    return last;
}
Node *root(Node *x){
    access(x);
    while(x->l) x = x->l;
    splay(x);
    return x;
}
void cut(Node *x){
    access(x);
    x->l->p = x->l = 0;
    update(x);
}
void link(Node *x, Node *y){
    access(x); access(y);
    x->l = y;
    y->p = x;
    update(x);
}
Node *lca(Node *x, Node *y){
    access(x);   return access(y);
}
int depth(Node *x){
    access(x);   return x->sz - 1;
}
class LinkCut{
    Node *x;

    public:
    LinkCut(int n){
        x = new Node[n];
        for(int i = 0; i < n; i++){
            x[i].label = i;
            update(&x[i]);
        }
    }
    virtual ~LinkCut(){ delete[] x; }
    void link(int u, int v){ ::link(&x[u], &x[v]); }
    void cut(int u){ ::cut(&x[u]); }
    int root(int u){ return ::root(&x[u])->label; }
    int depth(int u){ return ::depth(&x[u]); }
    int lca(int u, int v){
        return ::lca(&x[u], &x[v])->label;
    }
};
```

TRIES

```
typedef struct{
    char kar;
    bool isWord;
    int parent;
    map<char,int> child;
}node;
char s[100];
node tree[2000001];
int idx, len, nn, ma;
void insert(int nodeIdx, int charIdx){
    if (tree[nodeIdx].child.count(s[charIdx])==0){
        tree[++idx].kar = s[charIdx];
        tree[idx].parent = nodeIdx;
        tree[idx].child.clear();
        tree[nodeIdx].child[s[charIdx]] = idx;
    }
    if (charIdx==len-1)
        tree[tree[nodeIdx].child[s[charIdx]]].isWord =1;
    else insert(tree[nodeIdx].child[s[charIdx]], \
        charIdx+1);
}
void traverse(int x,int depth){
    depth += tree[x].isWord;
    if (ma < depth) ma = depth, nn = x;
    for(auto i: tree[x].child) traverse(i.se,depth);
}
int main(){
    idx = 0;
    tree[0].kar=0;
    ma=-1;
    while (scanf("%s",s)){
        if (strcmp(s,".")==0) break;
        len = strlen(s);
        insert(0,0);
    }
    nn=0;
    traverse(0,0);
    int res[100];
    char ss[100];
    int dd=0;
    while (nn){
        ss[dd]= tree[nn].kar;
        res[dd] = tree[nn].isWord;
        dd++;
        nn = tree[nn].parent;
    }
    reverse(ss,ss+dd);
    reverse(res,res+dd);
    ss[dd] = 0;
    for(int i=0;i<dd;i++)  if (res[i]){
        char temp = ss[i+1];
        ss[i+1] = 0;
        printf("%s\n",ss);
        ss[i+1] = temp;
    }
    return 0;
}
Input  :   i if in input int integer output
Output :   i in int integer
// Another trie
#define MA 5000010
typedef struct Tnode{
    int counter;
    char p;
    int child[3];
}node;
char s[210];
node trie[MA];
int idx,len;
int res;
```

```
void insert(int trid,int id){
    trie[trid].counter++;
    if (id == len) return;
    if (trie[trid].child[s[id]-'0']==-1){
        idx++;
        trie[idx].p = s[id];
        trie[trid].child[s[id]-'0'] = idx;
    }
    insert(trie[trid].child[s[id]-'0'], id+1);
}
void calc(int idx,int length){
    res = max(res, trie[idx].counter * length);
    if (trie[idx].child[0]!=-1)
        calc(trie[idx].child[0],length+1);
    if (trie[idx].child[1]!=-1)
        calc(trie[idx].child[1],length+1);
}
int main(){
    int  t,n;
    scanf("%d",&t);
    while (t--){
        idx = 0;
        scanf("%d",&n);
        for(int i=0;i<MA;i++) {
            trie[i].counter = 0
            trie[i].child[0] = trie[i].child[1] =-1;
        }
        while (n--){
            scanf("%s",s);
            len = strlen(s);
            insert(0,0);
        }
        res = 0;
        calc(0,0);
        printf("%d\n",res);
    }
}
```

MO ALGORITHM

```
typedef pair<pii,int> iii;
int blocks;
int n,l,r,q,ans,answer[300100],cnt[1000100],a[300100];
vector<iii> v;

bool cmp(iii x, iii y){
    if(x.fi.fi/blocks == y.fi.fi/blocks)
        return x.fi.se<y.fi.se;
    return x.fi.fi/blocks < y.fi.fi/blocks;
}
void add(int position){
    cnt[a[position]]++;
    if(cnt[a[position]]==1) ans++;
}
void remove(int position){
    cnt[a[position]]--;
    if(cnt[a[position]]==0) ans--;
}
int main(){
    scanf("%d",&n);
    blocks = (int)sqrt(n);
    FOR(i,n) scanf("%d",&a[i]);
    scanf("%d",&q);
    FOR(i,q){
        scanf("%d %d",&l,&r); l--, r--;
        v.pb(mp(mp(l,r),i));
    }
    sort(v.begin(), v.end(), cmp);
    int currentL = 0, currentR = -1;
    FOR(i,(int)v.size()){
        l = v[i].fi.fi; r = v[i].fi.se;
        int idx = v[i].se;
        while(currentR < r) add(++currentR);
        while(currentR > r) remove(currentR--);
        while(currentL < l) remove(currentL++);
        while(currentL > l) add(--currentL);
        answer[idx] = ans;
    }
    FOR(i,q) printf("%d\n", answer[i]);
    return 0;
}
```

MISOF TREE

```
class KthTree {
    private:
        int **tree,nlv;
        int lv(int);
    public:
        KthTree(int);
        ~KthTree();
        void insert(int);
        void erase(int);
        int KthElement(int);
};
int KthTree::lv(int n, int ret = 0){
    while( (1<<ret)<=n ) ret++;
    return ret;
}
KthTree::KthTree(int n){
    this->nlv = this->lv( n ) + 1;
    tree=(int**)malloc(sizeof(int*)*(this->nlv));
    FOR(i,this->nlv){
tree[i] = (int*)malloc(sizeof(int)*(1<<(this->nlv-1)));
memset(tree[i],0,sizeof(int)*(1<<(this->nlv-1)));
    }
}
KthTree::~KthTree(){
    FOR(i,this->nlv) free(tree[i]);   free(tree);
}
void KthTree::insert(int n){
    FOR(i,this->nlv) tree[i][n]++, n>>=1;
}
void KthTree::erase(int n) {
    FOR(i,this->nlv) tree[i][n]--, n>>=1;
}
int KthTree::KthElement(int k) {
    int a = 0, b = (this->nlv)-1;
    while(b--) if(tree[b][a<<=1] < k) k -= tree[b][a++];
    return a;
}
```

HEAVY LIGHT DECOMPOSITION

```
class Tree{
    private:
        int num, idx, n, elder;
        int dat[50005<<2];
        //LCA parent
        int par[50005][25];
        int depth[50005];
        int hashh[50005];
        int chain[50005];
        int headd[50005];
        vi child[50005];
        int count[50005]; //count subtree
        //generate function
        inline void genDepth(int,int);
        inline void genHLD(int);
        //LCA func
        inline int getNPar(int,int);
        inline int getAnc(int,int);
        //segment tree RURQ
        inline void upd(int,int,int,int,int,int);
        inline int que(int,int,int,int);
    public:
        inline void clear(int);
        inline void push(int,int);
        inline void generate();
        inline void upd(int,int,int);
        inline int que(int);
};
inline void Tree::genDepth(int _node, int _depth=0){
    depth[_node]=_depth; count[_node]=1;
```

```
    for(auto x:child[_node]){
        if(x==par[_node][0])continue;
        par[x][0]=_node;
        genDepth(x,_depth+1);
        count[_node]+=count[x];
    }
}
inline void Tree::genHLD(int _node){
    hashh[_node]=++idx;
    chain[_node]=num;
    pii temp=mp(-1,-1);
    for(auto x:child[_node])
        if(count[x]>temp.se&&x!=par[_node][0])
            temp=mp(x,count[x]);
    if(temp.fi!=-1)genHLD(temp.fi);
    for(auto x:child[_node]){
        if(x==temp.fi||x==par[_node][0])continue;
        chain[x]=++num,headd[num]=x;
        genHLD(x);
    }
}
inline int Tree::getNPar(int _node, int _n){
    for(int j=0;j<15&&_n;j++,_n>>=1)
        if(_n&1)_node=par[_node][j];
    return _node;
}
inline int Tree::getAnc(int _u, int _v){
    //stabilize depth
    if(depth[_u]< depth[_v])
        _v=getNPar(_v,depth[_v]-depth[_u]);
    if(depth[_u] >depth[_v])
        _u=getNPar(_u,depth[_u]-depth[_v]);
    //u==v
    if(_u==_v)return _u;
    //search LCA
    FORD(j,20)
        if(depth[_u]>=(1<<j)&&par[_u][j]!=par[_v][j])
            _u=par[_u][j]; _v=par[_v][j];
    return par[_u][0];
}
void::upd(int idx,int a,int b,int l,int r,int val){
    //base case
    if(b<l||r<a)return;
    if(l<=a&&b<=r){ dat[idx]+=val; return; }
    //reccurence
    int mid=(a+b)>>1;
    upd(2*idx,a,mid,l,r,val);
    upd(2*idx+1,mid+1,b,l,r,val);
}
inline int Tree::que(int idx, int a, int b, int pos){
    //base case
    if(b<pos||pos<a)return 0;
    if(pos<=a&&b<=pos)return dat[idx];
    //reccurence
    int mid=(a+b)>>1;
    if(pos<=mid)return dat[idx]+que(2*idx,a,mid,pos);
    return dat[idx]+que(2*idx+1,mid+1,b,pos);
}
//public
inline void Tree::clear(int _n){
    n=_n;
    FOR(i,_n+3){
        FOR(j,23)par[i][j]=0;
        child[i].clear();
        hashh[i]=depth[i]=count[i]=0;
        chain[i]=headd[i]=-1;
    }
    FOR(i,_n<<2)dat[i]=0;
    num=idx=elder=0;
}
inline void Tree::push(int _u, int _v){
    child[_u].pb(_v);   child[_v].pb(_u);
}
inline void Tree::generate(){
    //generate LCA
    genDepth(elder);
    REP(j,1,20) REP(i,0,n-1)
        par[i][j]=par[ par[i][j-1] ][j-1];
    //generate HLD :)
    chain[elder]=++num; headd[num]=elder; genHLD(elder);
}
inline void Tree::upd(int _u, int _v, int _val){
    int anc = getAnc(_u,_v);
    while(1){ // run from U
        int fron=headd[chain[_u]];
        if(chain[anc]==chain[_u])fron=anc;
        upd(1,1,n,hashh[fron],hashh[_u],_val);
        if(chain[anc]==chain[_u])break;
        _u=par[fron][0];
    }
    while(1){ // run from V
        int fron=headd[chain[_v]];
        if(chain[anc]==chain[_v])fron=anc;
        upd(1,1,n,hashh[fron],hashh[_v],_val);
        if(chain[anc]==chain[_v])break;
        _v=par[fron][0];
    }
    //substract anc
    upd(1,1,n,hashh[anc],hashh[anc],-_val);
}
inline int Tree::que(int pos){
    return que(1,1,n,hashh[pos]);
}
```

LIS (LONGEST INCREASING SUBSEQUENCE)

```
vi indi,arr((int)1e6,0); arr[0]=INF;
int cnt=0;
FOR(i,n){
    int lo=0, hi=cnt, mid;
    while(lo<=hi){
        mid=(lo+hi)/2;
        if(arr[mid]>=lst[i]) hi=mid-1;
        else lo=mid+1;
    }
    arr[lo]=lst[i];
    indi.pb(lo);
    if(cnt<lo) cnt=lo;
}
vi ans; int idx=n-1;
while(cnt>0){
    if(inid[idx]==cnt){
        ans.pb(lst[idx]);
        cnt--;
    }
    idx--;
} reverse(ans.begin(), ans.end());
```

CYCLIC LCS (LONGEST COMMON SUBSEQUENCE)

```
// O(nm)          https://arxiv.org/pdf/1208.0396v3.pdf
const int N=4000+10;
int dp[N][N], from[N][N];
int eq(int a, int b){ return s[(a-1)%n]==t[(b-1)%m]; }
int clcs(){
    REP(i,0,n<<1) REP(j,0,m) dp[i][j] = from[i][j] = 0;
    REP(i,0,n<<1) REP(j,0,m){
        dp[i][j] = 0;
        if(j && dp[i][j-1] > dp[i][j])
            dp[i][j] = dp[i][j - 1],from[i][j] = 0;
        if(i&&j && eq(i,j) && dp[i-1][j-1]+1>dp[i][j])
            dp[i][j] = dp[i-1][j-1]+1,from[i][j] = 1;
        if(i && dp[i-1][j] > dp[i][j])
            dp[i][j] = dp[i-1][j],from[i][j] = 2;
    }
    int ret = 0;
    FOR(i,n){
        ret = max(ret,dp[i+n][m]);
        int x = i+1, y = 0;
        while(y <= m && from[x][y] == 0) y++;
        for(; y <= m && x <= n*2; x++){
            from[x][y] = 0; dp[x][m]--;
```

```
            if(x == n*2) break;
            for(; y <= m; y++){
                if(from[x+1][y] == 2) break;
                if(y+1 <= m && from[x+1][y+1] == 1){
                    y++; break;
                }
            }
        }
    }
    return ret<<1;
}
```

LCIS (LONGEST COMMON INCREASING SUBSEQUENCE)

```
typedef long long LL;
const int INF = 1000000000;
const LL INFLL = (LL)INF*(LL)INF;
inline int getint(){ int x; scanf("%d",&x); return x; }
const int MAXN = 512;
int A[MAXN]; int na;
int B[MAXN]; int nb;
int prev[MAXN][MAXN];
LL best[MAXN][MAXN];
int pos[MAXN][MAXN];
inline void printPath(int path) {
    if ( prev[path>>10][path&1023] == -1 )
        printf("%d",A[path>>10]);
    else {
        printPath(prev[path>>10][path&1023]);
        printf(" %d",A[path>>10]);
    }
}
int main() {
    REP(i,na=getint()) A[i]=getint();
    REP(i,nb=getint()) B[i]=getint();
    REP(i,nb) REP(j,nb) best[i][j] = INFLL;
    REP(i,na) REP(j,nb) prev[i][j] = -1;
    REP(i,na) {
        int x = -1, p = 0;
        REP(j,nb) {
            if ( A[i] == B[j] ) {
                int y = p;
                while(best[j][y]<A[i]) y++;
                best[j][y] = A[i];
                pos[j][y] = (i<<10)+j;
                if ( y > 0 ) prev[i][j] = pos[j][y-1];
                x = p = y;
            } else {
                if(x!=-1 && best[j-1][x] < best[j][x]){
                    best[j][x] = best[j-1][x];
                    pos[j][x] = pos[j-1][x];
                } else x=-1;
            }
        }
    }
    int len=-1;
    FORD(i,nb-1,0) if(best[nb-1][i]!=INFLL){
        len = i; break; }
    printf("%d\n",len+1);
    if ( len >= 0 ) printPath(pos[nb-1][len]);
    puts("");
    return 0;
}
```

DP Buka String & Lipat String

```
struct tipetipe {int min, nkurung;};
int n;
string buka( int& i, string s ) {
    int len=s.length(), angka=0;   string tmp;
    while ( i<len && s[i] != ')' ) {
        if ( s[i]>='0' && s[i]<='9' )
            angka = angka*10 + (s[i]-'0');
        else if ( s[i] == '(' ) {
            i++;
            string tmp2 = buka( i, s );
            REP(i,angka) tmp += tmp2;
            angka = 0;
        } else tmp = tmp+s[i];
        i++;
    }
    return tmp;
}
tipetipe memo[305][305];
int path[305][305][4];
string s2;
tipetipe hohoho( int l, int r ) {
    //printf( "%d %d\n", l, r );
    //fflush(stdin);   getchar();
    if ( memo[r][l].min != -1 ) return memo[r][l];
    tipetipe ret;
    ret.min = r-l+1;
    ret.nkurung = 0;
    path[l][r][0] = 0;
    FOR( i,1,(r-l+1)/2 ) if ( (r-l+1)%i == 0 ) {
        string t1 = s2.substr( 0+l, i );
        string t2 = s2.substr( i+l, i );
        int ni=1, j=i+l;
        while ( t1 == t2 && j<=r ) {
            ni++; j+=i;
            t2 = s2.substr( j, i );
        }
        if ( ni == (r-l+1)/i ) {
            int dig=0;
            int nii=ni;
            while (ni>0) dig++, ni/=10;
            tipetipe tmp;
            tmp = hohoho(l,l+i-1);
            tmp.min += dig+2;
            tmp.nkurung += 2;
            if ( tmp.min < ret.min ) {
                ret = tmp;
                path[l][r][0] = 1;
                path[l][r][1] = nii;
                path[l][r][2] = i;
            }
            else if(tmp.min==ret.min && tmp.nkurung < \
            ret.nkurung){
                ret = tmp;
                path[l][r][0] = 1;
                path[l][r][1] = nii;
                path[l][r][2] = i;
            }
        }
    }
    FOR( i,l,r-1 ) {
        tipetipe tmp, left, right;
        left = hohoho(l,i); right = hohoho(i+1,r);
        tmp.min = left.min + right.min;
        tmp.nkurung = left.nkurung + right.nkurung;
        if ( tmp.min < ret.min ) {
            ret = tmp;
            path[l][r][0] = 2;
            path[l][r][1] = i;
        }
        else if ( tmp.min == ret.min && tmp.nkurung < \
        ret.nkurung ) {
            ret = tmp;
            path[l][r][0] = 2;
            path[l][r][1] = i;
        }
    }
    //printf("l->%d   r->%d   res->%d  path->%d\n", \
    l,r,ret,path[l][r][0] );
    return memo[r][l] = ret;
}
string search_path( int l, int r ) {
    string ret;
    if(path[l][r][0] == 0) FOR( i,l,r ) ret += s2[i];
    if ( path[l][r][0] == 1 ) {
        char tmpx[10];
        sprintf( tmpx, "%d", path[l][r][1] );
```

```
        ret += tmpx;
        ret += "("+search_path(l,l+path[l][r][2]-1)+")";
    }
    if ( path[l][r][0] == 2 ) {
        ret += search_path( l, path[l][r][1] );
        ret += search_path( path[l][r][1]+1, r );
    }
    return ret;
}
int main() {
    cin >> n;
    REP( i,n ) {
        string s1;
        cin >> s1;
        int z = 0;
        s2 = buka(z,s1);
        memset(memo,-1,sizeof(memo));
        hohoho( 0, s2.length()-1 );
        cout << s2 << endl;
        cout << search_path( 0, s2.length()-1 ) << endl;
    }
    return 0;
}
// input
2(haue)yueuyeuue3(xxzz)zzxx
hauehaueyueuyeuuexxzzxxzzxxzzzzxx
2(haue)yueuyeuue3(xxzz)zzxx
```

MATRIX CHAIN MULTIPLICATION

```
#define MAX 100
#define inf 2000000000
int p[MAX+1];
int m[MAX+1][MAX+1],s[MAX+1][MAX+1];

void MCM(int n) {
    int l, j, i, k, q;
    for(i=0;i<=n;i++)m[i][i]=0;
    for(l=2;l<=n;l++)  for(i=1;i<=n-l+1;i++) {
        j=i+l-1;   m[i][j]=inf;
        for(k=i;k<=j-1;k++) {
            q=m[i][k]+m[k+1][j]+p[i-1]*p[k]*p[j];
            if(q < m[i][j])  m[i][j] = q,s[i][j] = k;
        }
    }
}
void print_MCM(int i,int j) {
    if(i == j) printf("A%d", i);
    else {
        printf("(");
        print_MCM(i,s[i][j]);
        printf(" x ");
        print_MCM(s[i][j]+1,j);
        printf(")");
    }
}
int main(){
    int n, i, j, cs = 1;
    while( scanf("%d", &n ) == 1 ) {
        if( n == 0 ) break;
        for(i=0;i<n;i++) scanf("%d %d", &p[i], &p[i+1]);
        printf("Case %d: ", cs++ );
        MCM( n );
        print_MCM( 1, n  );
        puts("");
    }
    return 0;
}
```

MIN CUT POLYGON TRIAGULATION

```
inline int getCost(pii a, pii b, pii c){
    return getDist2(a,b)+getDist2(b,c)+getDist2(c,a);
}
int minCutPolyTri(){
    int res=0, dp[105][105];
    FOR(i,103) FOR(j,103)dp[i][j]=0;
    FOR(i,n)res+=getDist2(dat[i],dat[(i+1)%n]);
    //DP
    FOR(xx,n) for(int i=0,j=xx; j<n; i++,j++){
        dp[i][j]=(j<i+2)?0:INF;
        REP(k,i+1,j-1)dp[i][j] = min(dp[i][j],dp[i][k] \
            +dp[k][j]+getCost(dat[i],dat[j],dat[k]));
    }
    return (dp[0][n-1]-res)>>1;
}
```

KADANE

```
// Kadane 2D
int N, t = 0, a[100][100], pr[100];
int S = 1<<31, s = 0, k, l, x1=0,x2=0,y1=0,y2=0,j;
FOR(z,N){
    FOR(i,N) pr[i] = 0;
    REP(i,z,N-1) {
        t = j = k = l = 0, s = 1<<31;
        FOR(i,n) {
            pr[i] = pr[i] + a[x][i];
            t = t + pr[i];
            if( t > s) s = t, k = i,l = j;
            if( t < 0 ) t = 0, j = i + 1;
        }
        if( s > S) S = s, x1 = x, y1 = k, x2 = z, y2=l;
    }
}
cout << x1 <<" "<< y1 <<" "<< x2 <<" " << y2 <<endl;
cout << S;
// 1D
k = 0; l = 0; s = -INF; t = 0; j = 1;
REP(i,1,n){
    t += a[i];
    if(t > s) k = j, l = i, s = t;
    if(t < 0) t = 0, j = i+1;
}
```

TOPOLOGICAL SORT

```
void topoSort(int v) {
    vis[v] = 1;
    for(int i=0; i<ad[v].size(); i++)
        if ( !vis[ad[v][i]] ) topoSort(ad[v][i]);
    topoSorted.push_back(v);
}
int main() {
    topoSorted.clear();
    REP(i,N) if ( !vis[i] ) topoSort(i);
    reverse(topoSorted.begin(), topoSorted.end());
}
```

Kahn's Algorithm

```
void topoSortKahn() {
    // get all indegrees by traverse adj lists O(V+E)
    vi in_degree(V, 0);
    FOR(u,V) for(auto x:adj[u]) in_degree[x]++;
    queue<int> q; // queue all indegree == 0
    FOR(i,v) if(in_degree[i] == 0)q.push(i);
    int cnt = 0; // count vertex
    // Create a vector to store result
    // (A topological ordering of the vertices)
    vi res; // vector of topo sort result
    while (!q.empty()){
        int u = q.front();  q.pop();
        res.pb(u);
        for(auto x:adj[u]){
            if(--in_degree[x] == 0) q.push(x);
        }
        cnt++;
    }
    // Check if there was a cycle
    if(cnt != V){ puts("Cycle detected!");  return; }
    for(auto x:top_order) printf("%d ",x);  puts("");
}
```

K-th SHORTEST PATH

```
const int MAXN = 100;
const int MAXK = 40;
vector<PII> g[MAXN];
```

```
priority_queue<PII>pq;
int visit[MAXN],n,k;

int solve(int s,int t) {
    FILL(visit,0);
    pq.push(MP(0,s));
    int ret=0;
    while(!pq.empty()) {
        int awal = -pq.top().first;
        int f = pq.top().second; pq.pop();
        if (visit[f]>=k) continue;
        visit[f]++;
        if (f==t) ret+=awal;
        FOREACH(it,g[f]) if (visit[it->first]<k)
        pq.push(MP(-(awal+it->second),it->first));
    } return ret;
}
int main() {
    k = GI;
    n = GI;
    REP(nedge,GI) {
        int a = GI - 1,b = GI-1,C=GI;
        g[a].PB(MP(b,c));
        g[b].PB(MP(a,c));
    }
    printf("%d\n", solve(0,n-1));
    return 0;
}/* k = shortest path ke berapa.
n = jumlah vertex.
solve(i,j); jika ingin shortest path dari i ke j.
*/
```

MANHATTAN SPANNING TREE

```
const int INF = 1e+9;
struct Position{
    int x,y;
    int diff,sum;
    int id;
}p[150000];
vector<int> BIT;
vector<pair<pair<int,int>,int> > Raw_Edges;

bool sortpoint(Position A,Position B){
    if(A.x != B.x) return A.x > B.x;
    return A.y > B.y;
}
int GetNearest(int x) {
    int ret = -1;
    for (; x > 0; x -= (x&(-x)))
        if (BIT[x]!=-1 && (ret==-1 || p[BIT[x]].sum  \
        < p[ret].sum)) ret = BIT[x];
    return ret;
}
void Insert(int x, int id) {
    int maxi = BIT.size();
    for ( ; x < maxi; x += (x & (-x)))
        if (BIT[x]==-1 || (p[id].sum<p[BIT[x]].sum))
            BIT[x] = id;
}
void AddEdges(){
    vector<int> axis;
    axis.pb(-INF);
    for(int a=0;a<n;a++) axis.pb(p[a].diff);
    sort(axis.begin(),axis.end());
    axis.erase(unique(axis.begin(),axis.end()),    \
        axis.end());
    sort(p,p+n,sortpoint);
    BIT = vector<int>(axis.size(), -1);
    for(int a = 0; a < n; a++) {
        int pos = lower_bound(axis.begin(),axis.end(), \
                    p[a].diff) - axis.begin();
        int nearest = GetNearest(pos);
        if(nearest != -1) Raw_Edges.pb(mp(mp(p[a].id,  \
            p[nearest].id),p[nearest].sum - p[a].sum));
        Insert(pos,a);
    }
}
int main() {
    for(int a=0;a<n;a++) {
        scanf("%d %d",&p[a].x,&p[a].y);
        p[a].sum = p[a].x + p[a].y;
        p[a].diff = p[a].x - p[a].y;
        p[a].id = a;
    }
    for(int a=0;a<4;a++) {
        for(int b=0;b<n;b++) {
        if(a==1 || a==3) {
            swap(p[b].x,p[b].y);
            p[b].diff = p[b].x - p[b].y;
        }
        if(a==2) {
            p[b].x = -p[b].x;
            p[b].sum = p[b].x + p[b].y;
            p[b].diff = p[b].x - p[b].y;
        }
        }
        AddEdges();
    }//DO MST
}
```

DIRECTED MST

```
int con[MAXN][MAXN], in[MAXN];
int vis[MAXN], active[MAXN], n;
// vis state: 0:unvisited, 1:visited, 2:cycle, 3:active
int cycle(int idx, int &res){
    if (vis[idx]==0) {
        vis[idx] = 3;
        vis[idx]=(in[idx]!=-1)?cycle(in[idx],res): 1;
    } else if (vis[idx]==3){
        set<int> pseudoNodes;
        int minCycle = INF;
        for (int j=idx; ; j=in[j]){
            pseudoNodes.insert(j);
            minCycle <?= con[in[j]][j];
            if (in[j]==idx) break;
        }
        int minCost = INF, bestParent;
        FOREACH(it,pseudoNodes){
            int j = *it; assert(active[j]);
            REP(i,n) if (active[i] && con[i][j]!=-1 &&
            !pseudoNodes.count(i)) {
            if (minCost > con[i][j] - con[in[j]][j]){
                minCost = con[i][j] - con[in[j]][j];
                bestParent = i;
            }
            con[i][j] -= con[in[j]][j] - minCycle;
            }
        }
        res += minCost;
        in[idx] = bestParent;
        FOREACH(it,pseudoNodes){
            int j = *it;
            REP(i,n) if (active[i] && i!=idx &&
                con[i][j]!=-1){
                    if (con[i][idx]==-1 ||
                        con[i][idx]>con[i][j])
                        con[i][idx] = con[i][j];
                }
            REP(i,n) if (active[i] && i!=idx &&
                con[j][i]!=-1){
                        if (in[i]==j) in[i] = idx;
                            if (con[idx][i]==-1 ||
                            con[idx][i]>con[j][i])
                        con[idx][i] = con[j][i];
            }
            if (j!=idx) active[j] = false;
        }
        vis[idx] = 2;
    }
    return vis[idx];
```

```
}
int dmst(int root){
    int res = 0;
    in[root] = -1;
    REP(i,n) con[i][root] = -1;
    REP(i,n) if (i!=root) {
        int minCost = INF;
        REP(j,n) if (con[j][i]!=-1) {
            if (con[j][i] < minCost) {
                minCost = con[j][i];
                in[i] = j;
            }
        }
        res += minCost;
    }
    memset(active,1,sizeof(active));
    while (true){
        memset(vis,0,sizeof(vis));
        bool hasCycle = cycle(root,res) == 2;
        REP(i,n) if (active[i] && !vis[i])
            hasCycle |= cycle(i,res) == 2;
        if (!hasCycle) return res;
    }
}
int dfs(int idx){
    if (vis[idx]) return 0; else vis[idx] = true;
    int ret = 1;
    REP(i,n) if (con[idx][i]!=-1) ret += dfs(i);
    return ret;
}
bool connected(int root){
    memset(vis,0,sizeof(vis));
    return dfs(root) == n;
}
int main(){
    int nTC,m,u,v,w;
    scanf("%d",&nTC);
    for (int TC=1; TC<=nTC; TC++){
        printf("Case #%d: ",TC);
        scanf("%d %d",&n,&m);
        memset(con,-1,sizeof(con));
        REP(i,m){
            scanf("%d %d %d",&u,&v,&w);
            con[u][v] = w;
        }
        if (!connected(0)) puts("Possums!");
        else printf("%d\n", dmst(0));
    }
}
```

BELLMAN FORD (Negative Cycle Detection)

```
int bellmanFord(int s) {
    int i, j;
    for (i = 0; i < n; ++i) d[i] = INF;
    d[s] = 0;
    FOR(i,n-1) FOR(j,e)
        if (d[edge[j].v] > d[edge[j].u] + edge[j].w)
            d[edge[j].v] = d[edge[j].u] + edge[j].w;
    FOR(j,e) if (d[edges[j].u]+edges[j].w<d[edges[j].v])
        return 0;
    return 1;
}
```

DINIC ALGORITHM

```
#define SET(p) memset(p, -1, sizeof(p))
#define CLR(p) memset(p, 0, sizeof(p))
#define i64 long long
const int INF = 0x7fffffff;
const int MAXN = 5005, MAXE = 60006;
int src, snk, nNode, nEdge;
int Q[MAXN], fin[MAXN], pro[MAXN], dist[MAXN];
int flow[MAXE], cap[MAXE], next[MAXE], to[MAXE];
inline void init(int _src, int _snk, int _n) {
    src = _src, snk = _snk, nNode = _n, nEdge = 0;
    SET(fin);
}
inline void add(int u, int v, int c) {
    to[nEdge] = v, cap[nEdge] = c, flow[nEdge] = 0,
next[nEdge] = fin[u], fin[u] = nEdge++;
    to[nEdge] = u, cap[nEdge] = c, flow[nEdge] = 0,
next[nEdge] = fin[v], fin[v] = nEdge++;
}bool bfs() {
    int st, en, i, u, v;
    SET(dist);
    dist[src] = st = en = 0;
    Q[en++] = src;
    while(st < en) {
        u = Q[st++];
for(i=fin[u]; i>=0; i=next[i]) {
v = to[i];if(flow[i] < cap[i] && dist[v]==-1) {dist[v]
= dist[u]+1;Q[en++] = v;}}}
    return dist[snk]!=-1;
}int dfs(int u, int fl) {
    if(u==snk) return fl;
    for(int &e=pro[u], v, df; e>=0; e=next[e]) {
        v = to[e];
        if(flow[e] < cap[e] && dist[v]==dist[u]+1) {
            df = dfs(v, min(cap[e]-flow[e], fl));
            if(df>0) {
                flow[e] += df;
                flow[e^1] -= df;
                return df;
            }
        }
    }return 0;
}
i64 dinitz() {
    i64 ret = 0, df;
    while(bfs()) {
        for(int i=1; i<=nNode; i++) pro[i] = fin[i];
        while(true) {
            df = dfs(src, INF);if(df){ret+=(i64)df;}
            else{ break;}}}return ret;
}
int main() {
    int n, e, u, v, c, i;
    scanf("%d%d", &n, &e);
    init(1, n, n);
    for(i=0; i<e; i++) {
        scanf("%d%d%d", &u, &v, &c);
        if(u!=v) add(u, v, c);
    }
    printf("%lld\n", dinitz());
    return 0;}
```

LOWEST COMMON ANCESTOR (SEG TREE)

```
const int MAXN = 200005;
const int INF = (int)1e9;

vi adj[maxn];
int n, q;
int euler[2*MAXN], level[2* MAXN], h[2* MAXN], id;
// h[i] = index of the first occurence of node i in E
(euler[i])
bool vis[maxn];
struct node { int level, id; } st[4*2* MAXN];

void eulerTour(int u, int p, int l) {
    euler[id] = u;
    level[id] = l;
    if (!vis[u]) vis[u] = 1, h[u] = id;
    id++;
    FOR(i,(int)adj[u].size()){
        int v = adj[u][i];
        if (v != p) {
            eulerTour(v, u, l+1);
            euler[id] = u;
            level[id] = l;
            id++;
        }
    }
```

```
}
void build(int p, int l, int r) {
    if (l == r) {
        st[p].level = level[l];
        st[p].id = l;
        return;
    }
    int mid = (l+r)/2;
    build(p*2, l, mid);
    build(p*2+1, mid+1, r);
    st[p].level = min(st[p*2].level, st[p*2+1].level);
    st[p].id = st[p*2].level<st[p*2+1].level ?
        st[p*2].id : st[p*2+1].id;
}
node query(int p, int l, int r, inti, int j) {
    if (l > j || r <i) {
        node ret;
        ret.level = INF;
        return ret;
    }
    if (l >= i&& r <= j) return st[p];
    int mid = (l+r)/2;
    node ret1, ret2;
    ret1 = query(p*2, l, mid, i, j);
    ret2 = query(p*2+1, mid+1, r, i, j);
    return (ret1.level < ret2.level)? ret1 : ret2;
}
int lca(int r, int u, int v) {
    int a = min(h[u], h[v]), b = max(h[u], h[v]);
    intlca_id = query(1, 1, 2*n-1, a, b).id;
    return euler[lca_id];
}

int main() {
    int r, u, v;
    scanf("%d", &n);
    for (inti = 0; i< n-1; i++) {
        scanf("%d %d", &u, &v);
        adj[u].pb(v);
        adj[v].pb(u);
    }
    memset(euler, 0, sizeofeuler);
    memset(level, 0, sizeof level);
    memset(h, 0, sizeof h);
    memset(vis, 0, sizeof vis);
    id = 1;
    eulertour(1, 1, 0);

    memset(st, 0, sizeofst);
    build(1, 1, 2*n-1);

    scanf("%d", &q);
    while (q--) {
        scanf("%d %d %d", &r, &u, &v);
        int lca1, lca2, lca3, ans;
        lca1 = lca(1, r, u);
        lca2 = lca(1, r, v);
        lca3 = lca(1, u, v);
        if (lca1 == lca2 && lca2 == lca3) ans = lca1;
        else if(lca1==lca2 && lca3!=lca1) ans = lca3;
        else if(lca1==lca3 && lca2!=lca1) ans = lca2;
        else if(lca2==lca3 && lca1!=lca2) ans = lca1;
        printf("%d\n", ans);
    }
    return 0;
}
```

Let LCA(r, u, v) be the LCA of u and v with respect to root r. To compute LCA(r, u, v), we can compute, for any fixed ro (usually let ro = 1),
LCA(ro, r, u); // LCA(1, r, u);
LCA(ro, r, v); // LCA(1, r, v);
LCA(ro, u, v); // LCA(1, u, v);
and take the "odd man out", i.e., if two are equal and the third are different, then take the third, else if they are all equal, take any one.

MIN COST MAX FLOW

```
int tc, R, C, flow, mf, s, t, costperflow;
int res[maxn][maxn], cost[maxn][maxn], dist[maxn],
p[maxn];
vector<int> adj[maxn];

bool bellman() {
    for (int i = s; i <= t; i++) {
        dist[i] = INF;
        p[i] = -1;
    }
    dist[s] = 0;
    queue<int>q; q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int i = 0; i < adj[u].size(); i++) {
            int v = adj[u][i], w = cost[u][v];
            if(dist[v] > dist[u] + w && res[u][v] > 0) {
                p[v] = u;
                dist[v] = dist[u]+w;
                q.push(v);
            }
        }
    }
    return dist[t] != INF;
}
void augment(int u, int minedge) {
    if (s == u) { flow = minedge; return; }
    else if (p[u] != -1) {
        augment(p[u], min(minedge, res[p[u]][u]));
        res[p[u]][u] -= flow;
        res[u][p[u]] += flow;
        costperflow += cost[p[u]][u];
    }
}
int main() {
    scanf("%d", &tc);
    for (int caseno = 1; caseno <= tc; caseno++) {
        scanf("%d %d", &R, &C);
        memset(res, 0, sizeof res);
        memset(cost, 0, sizeof cost);
        for (int i = 0; i < maxn; i++) adj[i].clear();
        s = 0; t = 2*R*C+1;
        for (int i = 1; i <= R*C; i++) {
            adj[s].pb(i);   res[s][i] = 1;
        }
        for (int i = R*C+1; i <= 2*R*C; i++) {
            adj[i].pb(t);   res[i][t] = 1;
        }
        char input[30];
        for (int i = 0; i < R; i++) {
            scanf("%s", input);
            for (int j = 0; j < C; j++) {
                int u = id(i, j), d=direction(input[j]);
                for (int k = 0; k < 4; k++) {
                    int x = i+dr[k], y=j+dc[k];
                    if (x<0 || y<0 || x>=R || y>=C)
                        continue;
                    int v = R*C + id(x, y);
                    int c = k==d? 0 : 1;
                    adj[u].pb(v); adj[v].pb(u);
                    res[u][v] = 1; res[v][u] = 0;
                    cost[u][v] = c; cost[v][u] = -c;
                }
            }
        }
        int ans = 0; mf = 0;
        while (bellman()) {
            flow = 0; costperflow = 0;
            augment(t, INF);
            ans += costperflow;
```

```
        }
        printf("Case #%d: %d\n", caseno, ans);
    }
    return 0;
}
```

MIN COST MAX FLOW (Multiple Edge)

```
typedef struct {  int v,flow,cost,cap; }node;
node parent [ MAX_N+1 ];
int dist[ MAX_N+1 ], from[ MAX_N+1 ], v, e;
vector<node>  adj[MAX_N+1];
int augment(int s,int t) {
   bool update;
   FOR(i,1,v) dist[i] = INF,from[i]=-1;
   dist[s] = 0;
   FOR(i,1,v-1) {
      //bellman ford
      update = false;
      FOR(j,1,v) REP(k,adj[j].size())
         if (adj[j][k].flow < adj[j][k].cap)   {
            int edgeCost = adj[j][k].cost;
            if(adj[j][k].flow < 0)edgeCost = -edgeCost;
            if(dist[adj[j][k].v] > dist[j] + edgeCost){
               dist[adj[j][k].v] = dist[j] + edgeCost;
               parent[adj[j][k].v] = adj[j][k];
               from[adj[j][k].v] = j;
               update = true;
            }
         }
      if (!update) break;
   }
   if (dist[t]==INF) return 0;
   int p = t,f = INF,pa;
   node temp;
   for(;from[p]!=-1;p=from[p]) {
      pa = from[p];
      temp = parent[p];
      f = min(f, temp.cap-temp.flow);
   }
   p = t;
   for(;from[p]!=-1;p=from[p])  {
      pa = from[p];
      REP(i,adj[pa].size()){
         if (adj[pa][i].v==p &&
adj[pa][i].cost==parent[p].cost){
            adj[pa][i].flow += f;
            break;
         }}
      REP(i,adj[p].size()){
         if (adj[p][i].v==pa && adj[p][i].cost ==
parent[p].cost){
            adj[p][i].flow-=f;
            break;
         }}
   }
   return f*dist[t];
}
void addedge(int v1,int v2,int c,int cap,int flow=0) {
   bool f = true;
   REP(i,adj[v1].size())
      if (adj[v1][i].v==v2 && adj[v1][i].cost==c) {
         adj[v1][i].cap+=cap;
         adj[v1][i].flow+=flow;
         f = false;
         break;
      }
   if (f) adj[v1].push_back((node){v2,flow,c,cap});
}
//Add undirected edge a<->b dengan cost c, kapasitas 1
addedge(a,b,c,1);
addedge(b,a,c,1);
directed edge IN(i) -> OUT(i) dengan cost a dan
kapasitas 1
addedge(IN(i),OUT(i),-a,1,0);
addedge(OUT(i),IN(i),a,1,1); //blocking flow
```

MAXFLOW (AdJ List)

```
const int maxv = 100, INF = (int)1e9;
typedef vector<int> vi;

int res[maxv][maxv], s, t, mf, f;
vi p, adj[maxv];

void augment(int v, intminedge) {
    if (v == s) { f = minedge; return; }
    else if (p[v] != -1) {
        augment(p[v], min(minedge, res[p[v]][v]));
        res[p[v]][v] -= f;
        res[v][p[v]] += f;
    }
}

int main() {
    mf = 0;
    while (1) { // BFS
        f = 0;
        vi dist(maxv, INF); dist[s] = 0;
        queue<int> q; q.push(s);
        p.assign(maxv, -1);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (u == t) break;
            int size = adj[u].size();
            for (inti = 0; i< size; i++) {
                int v = adj[u][i];
                if (dist[v]== INF && res[u][v] > 0) {
                    dist[v] = dist[u] + 1;
                    p[v] = u;
                    q.push(v);
                }
            }
        }
        augment(t, INF);
        if (f == 0) break;
        mf += f;
    }
    printf("%d\n", mf);
    return 0;
}
```

MIN CUT (Stoer Wagner)

```
int m[600][600], node[600];
int inset[600], d[600];
int reduce( int n ) {
    memset( inset, 0, sizeof(inset) );
    memset( d, 0, sizeof(d) );
    inset[ node[0] ] = 1;
    REP(i,1,n-1) d[node[i]] = m[node[0]][node[i]];
    int ltnode = 0;
    for( int _=1; _<n; _++ ) {
        int nd = -1;
        FOR(i, 1, n-1) {
            if ( inset[ node[i] ] ) continue;
            if ( nd == -1 ) nd = i;
            if ( d[ node[i] ] > d[ node[nd] ] ) nd = i;
        }
        inset[ node[nd] ] = 1;
        if ( _ == n -1 ) {
            REP(i,n) {
                m[ node[ltnode] ][ node[i] ] += m[
node[nd] ][ node[i] ];
                m[ node[i] ][ node[ltnode] ] = m[
node[ltnode] ][ node[i] ];
            }
            int ret = d[ node[nd] ];
            node[nd] = node[n-1];
            return ret;
        }
        ltnode = nd;
        REP(i,n) {
            if ( inset[ node[i] ] ) continue;
```

```
            d[ node[i] ] += m[ node[nd] ][ node[i] ];
        }
    }
}
int mincut( int n ) {
    for( int i=0; i<n; i++ ) node[i] = i;
    int ans = ( 1 << 30 );
    while( n > 1 ) {
        int mcut = reduce( n );
        if ( mcut < ans ) ans = mcut;
        n--;
    }
    return ans;
}
int main() {
    int N, M;
    while( scanf("%d %d", &N, &M) != EOF ) {
        memset( m, 0, sizeof(m) );
        for( int i=0, a, b, c; i<M; i++ ) {
            scanf("%d %d %d", &a, &b, &c);
            m[a][b] += c;
            m[b][a] += c;
        }
        printf("%d\n", mincut( N ) );
    } }
```

EULERIAN TOUR CHECKING

```
// Every edge is visited exactly once
int N, M, d[250], di[250];
int m[205][205], pre[205];
int vis[205];

int flow( int f, int si) {
    queue< int > q;
    q.push( f );
    memset( vis, 0, sizeof(vis) );
    memset( pre, -1, sizeof(pre) );
    while( !q.empty() ) {
        int now = q.front();
        q.pop();
        if ( vis[now] == 2 ) continue;
        vis[now] = 2;
        if ( now == si ) break;
        for( int i=1; i<=N+1; i++ ) {
            int next = i;
            if ( vis[next] ) continue;
            if ( m[now][next] > 0 ) {
                vis[next] = 1;
                pre[next] = now;
                q.push( next );
            }
        }
    }
    int nt = si, ret = ( 1 << 30 );
    while( pre[nt] != -1 ) {
        int bf = pre[nt];
        ret = min( ret, m[bf][nt] );
        nt = bf;
    }
    if ( ret >= ( 1 << 30 ) ) return 0;
    nt = si;
    while( pre[nt] != -1 ) {
        int bf = pre[nt];
        m[bf][nt] -= ret;
        m[nt][bf] += ret;
        nt = bf;
    }
    return ret;
}

int main() {
    int ntc;
    scanf("%d", &ntc);
    while( ntc-- ) {
        scanf("%d %d", &N, &M);
        memset( d, 0, sizeof(d) );
        memset( m, 0, sizeof(m) );
        for( int i=0, a, b, c; i<M; i++ ) {
            scanf("%d %d %d", &a, &b, &c);
            d[a]++, d[b]--;
            if ( !c ) m[a][b]++;
        }
        int can = 1;
        for( int i=1; can && i<=N; i++ ) {
            int x = ( d[i] < 0 ) ? -d[i] : d[i];
            if ( x%2 == 1 ) can = 0;
            d[i] = d[i] / 2;
        }
        if ( can ) {
            int tflow = 0;
            for( int i=1; i<=N; i++ ) {
                if ( d[i] < 0 ) m[i][N+1] = -d[i];
                else if ( d[i] > 0 ) {
                    m[0][i] = d[i];
                    tflow += d[i];
                }
            }
            int tans, ans = 0;
            while((tans = flow(0,N+1)) > 0) ans += tans;
            can = can && ( ans == tflow );
        }
        puts( (can) ? "possible" : "impossible" );
    }
    return 0;
}
```

MAX BIPARTITE MATCHING

```
vi match, vis;

int aug(int l){
    if (vis[l]) return 0;
    vis[l] = 1;
    for(auto r: AdjList[l])
        if(match[r] == -1 || aug(match[r]))
            match[r] = l; return 1;
    return 0;
}

// inside int main()
int MCBM = 0;
match.assign(V, -1);
FOR(i,n) vis.assign(n,0), MCBM += aug(i);
printf("Found %d matchings\n", MCBM);
```

MINIMUM VERTEX COVER

```
vi AdjList[100010];
int memo[100010][2];

int solve(int idx, int flag, int prev){
    int ans=0;
    if(memo[idx][flag] != -1) return memo[idx][flag];
    if(AdjList[idx].size()==1 && AdjList[idx][0]==prev)
        return memo[idx][flag] = flag;
    //reccurence
    if(flag==0){
        ans=0;
        for(auto j: AdjList[idx]) if(prev != j)
            ans += solve(j,1,idx);
    }
    if(flag==1){
        ans=1;
        for(auto j: AdjList[idx]) if(prev != j)
            ans += min(solve(AdjList[idx][j],1,idx),
                       solve(AdjList[idx][j],0,idx));
    }
    return memo[idx][flag] = ans;
}

int main(){
    int n,a,b;
    scanf("%d",&n);
```

```
    for(int i=0;i<n-1;i++){
        scanf("%d %d",&a,&b);
        AdjList[a].pb(b);
        AdjList[b].pb(a);
    }
    memset(memo,-1,sizeof(memo));
    int ans = solve(1,0,-1);
    memset(memo,-1,sizeof(memo));
    int ans2 = solve(1,1,-1);
    printf("%d\n",min(ans,ans2));
    return 0;
}
```

ARTICULATION POINT & BRIDGE

```
Vi dfs_num, dfs_low, dfs_parent;
int dfs_counter=0, rootChildren, dfs_root;
void articulationPoint(int u){
   dfs_num[u] = dfs_low[u] = dfs_counter++;
   for(int i=0;i<AdjList[u].size();i++){
    int v = AdjList[u][i];
    if(dfs_num[v]==INF){
        dfs_parent[v]=u;
        if(u==dfsRoot) rootChildren++;
       articulationPoint(v);
        if(dfs_low[v]>=dfs_num[u])
            flag[u]=true; //ini articulation Point
        if(dfs_low[v] > dfs_num[u])
            // ini adalah bridge(Edge(u,v))
        dfs_low[u]=min(dfs_low[u],dfs_low[v]);
    }
    else if(v!=parent[u])
        dfs_low[u]=min(dfs_low[u],dfs_num[v]);
   }
}
//inside int main
dfs_counter=0; dfs_num.assign(V,INF);
dfs_low.assign(V,0); dfs_parent(V,0);
memset(flag,false,sizeof(flag));
for(int i=0;i<V;i++){
   if(dfs_num[i]==INF){
    dfs_root = i; rootChildren=0;
    articulationPoint(i);
    flag[i]=(rootChildren>1);
   }
}
```

TARJAN ALGORITHM

```
vi dfs_num, dfs_low, S, visit;
void tarjanSCC(int u){
   dfs_low[u] = dfs_num[u] = dfs_counter++;
   S.push_back(u); visited[u]=1;
   for(int j=0;j<AdjList[u].size();j++){
    int v = AdjList[u][j];
    if(dfs_num[v]==UNVISITED) tarjanSCC(v);
    if(visit[v])
        dfs_low[u]=min(dfs_low[u],dfs_low[v]);
   }
   if(dfs_low[u]==dfs_num[u]){
    printf("%SCC %d:",++numSCC);
    while(1){
        int v = S.back(); S.pop_back(); visit[v]=0;
        printf(" %d",v);
        if(u==v) break;
      }
   }
}
//inside int main
dfs_num.assign(V,UNVISITED); dfs_low.assign(V,0);
visit.assigned(V,0); dfs_counter=numSCC=0;
for(int i=0;i<V;i++)
   if(dfs_num[i]==UNVISITED) tarjanSCC(i);
```

KMP ALGORITHM

```
char s[MAX+5], a[MAX+5]; int pos[MAX+5];
inline int KMP(int k){
    int la=strlen(a), ls=strlen(s), temp, res;
    temp=pos[0]=-1;
    FOR(i,la){
        while(pot>=0&&a[i]!=a[pot])temp=pos[temp];
        pos[i+1]=++temp;
    }
//  FOR(i,la+5)printf(" %d ",pos[i]); puts("");
    temp=res=0;
    FOR(i,ls){
        while(temp>=0&&s[i]!=a[temp])temp=pos[temp];
        if(++temp==la)pot=pos[pot],return 1; //found
    }
    return 0; //not found
}
```

SUFFIX ARRAY

```
int dat[100005][25];
pair<pii,int> ret[100005];

void sparse(char *s, int n){
    FOR(i,100003) FOR(j,23)dat[i][j]=-1;
    FOR(j,20){
        //copy data
        ret[0]=mp(mp(-1,-1),-1);
        FOR(i,n){
            ret[i+1].se=i;
            if(j==0)ret[i+1].fi=mp((int)s[i],-1);
            else{
                int mask=(i+(1<<(j-1)))%n;
                ret[i+1].fi.fi=dat[i][j-1];
                ret[i+1].fi.se=dat[mask][j-1];
            }
        }
        //sort
        sort(ret+1,ret+1+n);
        //iterate
        int pos=0;
        REP(i,1,n){
            if(ret[i-1].fi==ret[i].fi)
                dat[ret[i].se][j]=pos;
            else dat[ret[i].se][j]=++pos;
        }
    }
}
int main(){
    int t,n; char s[200005];
    scanf("%d",&t);
    while(t--){
        scanf("%d %s",&n,s); sparse(s,n);
        //iterate
        int res=0;
        FOR(i,n){
            //compare with pos
            int posa=i, posr=res;
            FORD(j,20) if(n&(1<<j)){
                if(dat[posa][j]==dat[posr][j]){
                    posa=(posa+(1<<j))%n;
                    posr=(posr+(1<<j))%n;
                    continue;
                }
                if(dat[posa][j]< dat[posr][j])res=i;
                break;
            }
        }
        //print
        printf("%d\n",res);
    }
    return 0;
}
```

ENHANCED SUFFIX ARRAY

```
#define MAX_N 100000
typedef struct  {  int a[2],idx; }ttemp;
inline bool cmp(const ttemp &a,const ttemp &b) {
   return (a.a[0]==b.a[0]) ? (a.a[1] < b.a[1]) :
(a.a[0] < b.a[0]);
}
```

```
ttemp L[MAX_N],temp[MAX_N];
int step,p[20][MAX_N],lt,lq, a[MAX_N+1];
inline void rasort(ttemp x[],int n) {
   int k=-1;
   memset(a,0,sizeof(a));
   REP(i,n) a[x[i].a[1]+1]++, k=max(k,x[i].a[1]+1);
   REP(i,1,k) a[i]+=a[i-1];
   FORD(i,n) temp[--a[x[i].a[1]+1]]=x[i];
   memset(a,0,sizeof(a));
   k = -1;
   REP(i,n)a[temp[i].a[0]+1]++,k=max(k,temp[i].a[0]+1);
   REP(i,1,k) a[i]+=a[i-1];
   FORD(i,n) x[--a[temp[i].a[0]+1]] = temp[i];
}
inline void generate(char s[],int p[][MAX_N]) {
   int cnt;
   FOR(i,lt) p[0][i] = s[i];
   for( step=1,cnt=1;(cnt>>1) < lt; step++,cnt<<=1) {
      FOR(i,lt) {
         L[i].a[0] = p[step-1][i];
         L[i].a[1] = ( i + cnt < lt ? p[step-1][i +
cnt] : -1 );
         L[i].idx = i;
      }
      rasort(L,lt);
      //sort(L,L+lt);  n log^2 n
      FOR(i,lt)
         p[step][L[i].idx] = i>0 && L[i].a[0] == L[i-
1].a[0] && L[i].a[1] == L[i-1].a[1] ? p[step][L[i-
1].idx]:i ;
   }
   step--;
}
char pattern[1100],text[100010];
//compare pattern suffix[idx]
inline int compare(int idx) {
   int i = 0;
   for(;i<lq && idx < lt;i++,idx++) {
      if (pattern[i] > text[idx]) return -1;
      if (pattern[i] < text[idx]) return 1;
   }
   return (i == lq ? 0 : -1 );
}
inline int lcp(int x,int y) {
   if (x==y) return lt-x;
   int res = 0;
   for(int k = step-1; k>=0 && x<lt && y<lt; k--)
      if (p[k][x]==p[k][y]){
         x+=(1<<k),y+=(1<<k),res+=(1<<k);}
   return res;
}
//cari many pattern pada text yang panjang, preprocess
text
int L = 0,R = lt-1,mid, found = false;
while ( L <= R && !found) {
      mid = (L+R)>>1;
      int temp = compare(arr[mid]);
      if (temp==0) found = 1;
      else if (temp==1) R = mid-1;
      else L = mid+1;
}
//Longest palindrome
S = S + rev(S), edit LCP
for(int i=0;i<lt/2-1 ;i++){
   res >?= lcp(lt-i-1,i+1);
   if (i)  res>?= 1 + lcp(lt-i,i+1);
}
//Distinct substring
if (lt&1) res = ((lt+1)>>1)*lt;
else res = (lt>>1) * (lt+1);
REP(i,lt) arr[p[step-1][i]] = i;
REP(i,lt-1)  res -= lcp(arr[i],arr[i+1]);
//longest palindrome, maximum K change
inline int longpal(int left,int right,int k){
   int temp = 0,a;
   for(int j=0;j<=k;){
      a = lcp(right,left);
      temp +=2*a;
      left +=a;
      right+=a;
      if (a==0) {
         if (j==k) break;
         temp+=2;
         left++,right++;
         j++;
      }
      if (left>= lt || right >= lt/2) break;
   }
   return temp;
}
```

EDIT DISTANCE

```
int dp[MAXN][MAXN];
char s1[MAXN],s2[MAXN];
int main() {     int len1,len2;
    while(scanf("%d", &len1)!=EOF) {
        if (len1>0) scanf("%s", s1);
        scanf("%d", &len2);
        if (len2>0) scanf("%s", s2);
        bool one = (len1==0)?1:0, two = (len2==0?1:0;
        FILL(dp,0);
        dp[0][0] = (s1[0]==s2[0]) ? 0 : 1;
        FOR(i,1,len1-1) dp[i][0] = (s1[i]==s2[0]) ?    \
            i : MIN(i+1,dp[i-1][0]+1);
        FOR(i,1,len2-1) dp[0][i] = (s1[0]==s2[i]) ?    \
            i : MIN(i+1,dp[0][i-1]+1);
        FOR(i,1,len1-1) {
            FOR(j,1,len2-1) {
                dp[i][j] = INF;
                if(s1[i]==s2[j])dp[i][j]<?=dp[i-1][j-1];
                dp[i][j] = MIN(dp[i][j],MIN(dp[i-1][j] +
1, MIN(dp[i][j-1] + 1,dp[i-1][j-1] + 1)));
        }  }
        printf("%d\n", (one^two) ? (max(len1,len2)) :
dp[len1-1][len2-1]);
    }   }
```

K-th PERMUTATION

```
string kthpermute(LL k, string s) {
    int len = s.length();
    char ss[len+2];
    sprintf(ss, "%s", s.c_str());
    LL fac = 1LL;
    FOR(i,2,len-1) fac = LL(i) * LL(fac);
    REP(i, len-1) {
        int tidx = ( k / fac ) % ( len - i );
        char ch = ss[ i + tidx ];
        for(int j = i+tidx; j > i; j--) ss[j] = ss[j-1];
        ss[i] = ch;
        fac = fac / LL( (len - i - 1) );
    }
    return (s = ss);
}
int main() {
    string s = "abcdefghijklmnopqrst";
    printf("%s\n", kthpermute(123456789987654321LL,
s).c_str() );
    return 0;
}
```

Soal TIMO 2010

```
const LL mod = 1000000007;
LL  nck[100][100];
LL  memo[10][100];
int arr[10];
LL f(int x, int m) {
    if ( m ==  0 ) return 1;
    if ( x == 10 ) return 0;
    if ( memo[x][m] != -1 ) return memo[x][m];
    LL &ret = memo[x][m] = 0;
```

```
    FOR(i,0,min(m,arr[x])) ret = (ret + (f(x+1,m-i) *
nck[m][i]) % mod) % mod;
    return ret;
}
int main(){
    nck[0][0] = 1;
    FOR(i,1,99) {
        nck[i][0] = nck[i][i] = 1;
        FOR(j,1,i-1) nck[i][j] = (nck[i-1][j] + nck[i-
1][j-1]) % mod;
    }
    int T;
    scanf( "%d", &T );
    while ( T-- ) {
        FOR(i,1,9) scanf( "%d", &arr[i] );
        int total = accumulate(arr+1,arr+10,0);
        LL ans = 0;
        FOR(k,1,9) if ( arr[k] != 0 ) {
            arr[k]--;
            memset(memo,-1,sizeof(memo));
            LL p = k;
            FOR(len,1,total) {
                ans = (ans + (f(1,len-1)*p)%mod) % mod;
                p   = (p * 10 + k) % mod;
            }
            arr[k]++;
        }
        printf( "%I64d\n", ans );
    }
}
// Problem statement: Given the number of available
// digits of digit 1 to 9, sum all possible numbers
// that can be generated by those available digits.
```

MEET IN THE MIDDLE

```
long sumsLowerBound(long[] A, long B) {
    int n = A.length;
    int sizeL = 1 << (n / 2),sizeR=1<<(n - n / 2);
long[] SumsL=new long[sizeL],SumsR=new long[sizeR];
    for (int i = 0; i < sizeL; ++i){
        for (int j = 0; j < n / 2; ++j){
            if ((i & (1 << j)) > 0){
                SumsL[i] += A[j];}}}
    for (int i = 0; i < sizeR; ++i){
        for (int j = 0; j < n - n / 2; ++j){
            if ((i & (1 << j)) > 0){
                SumsR[i] += A[j + n / 2];}}}
    Arrays.sort(SumsL);
    Arrays.sort(SumsR);
    int PL = 0, PR = sizeR - 1;
    long cur = Long.MIN_VALUE;
    while (PL < sizeL && PR >= 0) {
        if (SumsL[PL] + SumsR[PR] <= B) {
            cur = Math.max(cur, SumsL[PL++]+SumsR[PR]);
        else PR--;
    }
    return cur;
}
```

HUFFMAN CODES

```
#include <fstream>
using namespace std;
typedef long long ll;
int main() {
    priority_queue<ll, vector<ll>, greater<ll>> q;
    ifstream in("plank.in");
    ofstream out("plank.out");
    int N;
    in >> N;
    for (int i = 0; i < N; i++) {
        ll x;
        in >> x;
        q.push(x);
    }
    ll cost = 0;
    for (;;) {
        ll a = q.top();
        q.pop();
        if (q.empty()) break;
        ll b = q.top();
        q.pop();
        cost += a + b;
        q.push(a + b);
    }
    out << cost << "\n";
    return 0;
}
```

HUNGARIAN ALGORITHM

```
int cost[64][64], lvisit[64], rvisit[64];
int lvalue[64], rvalue[64], rmatch[64], n;
inline bool findmatch(int s) {
    lvisit[s] = true;
    REP(i,n) {
        if(rvisit[i]||lvalue[s]+rvalue[i]!=cost[s][i])
continue;
        rvisit[i] = true;
        if(rmatch[i]==-1 || findmatch(rmatch[i])) {
            rmatch[i] = s;
            return true;
        }
    }
    return false;
}
inline void normalize() {
    int down = INF;
    REP(i,n) {
        if(lvisit[i]) {
            REP(j,n) {
                if(!rvisit[j]) down =
min(down,lvalue[i]+rvalue[j]-cost[i][j]);
            }
        }
    }
    REP(i,n) {
        if(lvisit[i]) lvalue[i] -= down;
        if(rvisit[i]) rvalue[i] += down;
    }
}
inline int calc() {
    REP(i,n) {
        lvalue[i] = *max_element(cost[i],cost[i]+n);
        rvalue[i] = 0;
    }
    REP(i,n) rmatch[i] = -1;
    REP(i,n) {
        FILL(lvisit,0);
        FILL(rvisit,0);
        if(findmatch(i)) continue;
        i--;
        normalize();
    }
    int ret = 0;
    REP(i,n) ret += cost[rmatch[i]][i];
    return ret;
}
```

STABLE MARRIAGE

```
int m, n;
int L[max_man][max_woman],
R[max_woman][max_man];
int L2R[max_man], R2L[max_woman];
int p[max_man];
void stableMarriage() {
    static int p[128];
    memset(R2L, -1, sizeof(R2L));
    memset(p, 0, sizeof(p));
    for( int i = 0; i < m; i++ ) {
        int man = i;
        while( man >= 0 ) {
            int wom;
            while( 1 ) {
```

```
                wom = L[man][p[man]++];
                if( R2L[wom] < 0 || R[wom][man] >
R[wom][R2L[wom]] ) break;
            }
            int hubby = R2L[wom];
            R2L[L2R[man] = wom] = man;
            man = hubby;
        }
    }
}
```

CSOD

```
LL sum(LL a,LL b) {
    a--;return (b*(b+1)>>1) - (a*(a+1)>>1);
}
LL CSOD(LL n) {
    LL i,m=n>>1;
    LL out2=0,j=sqrt(n);
    for(int i=2;i<=j;i++) out2=(n/i)*i;
    for(i--;j>=1;j--) {
        out2 += (j*sum(i+1,n/j));
        i = (n/j);
    }
    out2 -= ((n*(n+1)>>1)-1);
    return out2;
}
```

BIG INTEGER

```
const int maxlength=1000;
class bigint{
public:
    // 1. Basic
    int oper,length,a[maxlength];
    bigint(int=0);
    ~bigint();
    void check();
    void operator=(bigint m);
    void operator=(int m);
    void operator=(char *s);
    int max(int a,int b);
    // 2. Relational operations
    bool operator<(bigint m);
    bool operator<=(bigint m);
    bool operator>(bigint m);
    bool operator>=(bigint m);
    bool operator==(bigint m);
    bool operator!=(bigint m);
    // 3. Arithmetic operations
    bigint operator-();
    bigint operator+(bigint m);
    void operator+=(bigint m);
    bigint operator-(bigint m);
    void operator-=(bigint m);
    bigint operator*(bigint m);
    bigint operator*(int m);
    void operator*=(bigint m);
    void operator*=(int m);
    bigint operator/(bigint m);
    bigint operator/(int m);
    void operator/=(bigint m);
    void operator/=(int m);
    bigint operator%(bigint m);
    bigint operator%(int m);
    void operator%=(bigint m);
    void operator%=(int m);
};
    // 4. Input/ Output
bool read(bigint &m);
void write(bigint m);
void swrite(char *s,bigint m);
void writeln(bigint m);
    // 5. Mathematical functions
bigint abs(bigint m);
bigint sqr(bigint m);
bigint sqrt(bigint m);
bigint gcd(bigint a,bigint b);
bigint lcm(bigint a,bigint b);
bigint max(bigint a,bigint b);
bigint min(bigint a,bigint b);
bigint fpow(bigint a,bigint b);
// B. Class Implementation
    // 1. Basic
bigint::bigint(int v){
    (*this)=v;
    this->check();
}
bigint::~bigint(){}
void bigint::check(){
    for (;length>0 && a[length]==0;length--);
    if (length==0)oper=1;
}
void bigint::operator=(bigint m){
    oper=m.oper;
    length=m.length;
    memcpy(a,m.a,maxlength*sizeof(int));
    this->check();
}
void bigint::operator=(int m){
    oper=(m>0)?1:-1;
    m=abs(m);
    memset(a,0,maxlength*sizeof(int));
    for (length=0;m>0;m=m/10000){
        a[++length]=m%10000;}
    this->check();
}
void bigint::operator=(char *s){
    int i,L;
    (*this)=0;
    if (s[0]=='-' || s[0]=='+') {
        if (s[0]=='-')oper=-1;
        L=strlen(s);
        for (i=0;i<L;i++) s[i]=s[i+1];
    }
    L=strlen(s);
    length=(L+3)/4;
    for (i=0;i<L;i++) {
        a[(L-i+3)/4]=a[(L-i+3)/4]*10+(s[i]-48);}
    this->check();
}
int bigint::max(int a,int b){return (a>b)?a:b;}
    // 2. Relational operations
bool bigint::operator<(bigint m) {
    if (oper!=m.oper) return oper<m.oper;
    if (length!=m.length){
        return oper*length<m.length*oper;}
    for (int i=length;i>=1;i--){
        if (a[i]!=m.a[i]){
            return a[i]*oper<m.a[i]*oper;}}
    return false;
}
bool bigint::operator<=(bigint m){return !(m<(*this));}
bool bigint::operator>(bigint m){return m<(*this);}
bool bigint::operator>=(bigint m){return !((*this)<m);}
bool bigint::operator==(bigint m){return (!((*this)<m))
&& (!(m<(*this)));}
bool bigint::operator!=(bigint m){return ((*this)<m) ||
(m<(*this));}
    // 3. Arithmetic Operations
bigint bigint::operator-(){
    bigint c=(*this);
    c.oper=-c.oper;
    c.check();
    return c;
}
bigint bigint::operator+(bigint m){
    if (m.length==0)return (*this);
    if (length==0)return m;
    if (oper==m.oper){
        bigint c;
        c.oper=oper;
```

```
        c.length=max(length,m.length)+1;
        for (int i=1,temp=0;i<=c.length;i++) {
        c.a[i]=(temp=(temp/10000+a[i]+m.a[i]))%10000;}
        c.check();
        return c;
    }
    return (*this)-(-m);
}
bigint bigint::operator-(bigint m){
    if (m.length==0)return (*this);
    if (length==0)return (-m);
    if (oper==m.oper){
        bigint c;
        if (abs(*this)>=abs(m)){
            c.oper=oper;
            c.length=length;
            for (int i=1,temp=0;i<=length;i++){
                c.a[i]=((temp=(-int(temp<0)+a[i]-
m.a[i]))+10000)%10000;}
            c.check();
            return c;
        }
        return -(m-(*this));
    }
    return (*this)+(-m);
}
bigint bigint::operator*(bigint m){
    bigint c;
    c.oper=oper*m.oper;
    c.length=length+m.length;
    for (int i=1;i<=m.length;i++){
        int number=m.a[i],j,temp=0;
        for (j=1;j<=length;j++){
            c.a[i+j-1]+=number*a[j];}
        if (i%10==0 || i==m.length){
            for (j=1;j<=c.length;j++){
    c.a[j]=(temp=(temp/10000)+c.a[j])%10000;}}
    }
    c.check();
    return c;
}
bigint bigint::operator*(int m){
    if (m<0)return -((*this)*(-m));
    if (m>100000)return (*this)*bigint(m);
    bigint c;
    c.length=length+2;
    c.oper=oper;
    int t=0;
    for (int i=1;i<=c.length;i++){
        c.a[i]=(t=(t/10000+a[i]*m))%10000;}
    c.check();
    return c;
}
bigint bigint::operator/(bigint m){
    if (m.length==0){
        printf("Division by zero.\n");
        exit(0);}
    if (abs(*this)<abs(m))return 0;
    bigint c,left;
    c.oper=oper/m.oper;
    m.oper=1;
    c.length=length-m.length+1;
    left.length=m.length-1;
    memcpy(left.a+1,a+length-
left.length+1,left.length*sizeof(int));
    for (int i=c.length;i>=1;i--){
        left=left*10000+a[i];
        int head=0,tail=10000,mid;
        while (head+1<tail){
            mid=(head+tail)/2;
            if (m*mid<=left)head=mid;
            else tail=mid;
        }
        c.a[i]=head;
        left-=m*head;
    }
    c.check();
    return c;
}
bigint bigint::operator/(int m){
    if (m<0)return -((*this)/(-m));
    if (m>100000)return (*this)/bigint(m);
    bigint c;
    c.oper=oper;
    c.length=length;
    int t=0;
    for (int i=c.length;i>=1;i--){
        c.a[i]=(t=(t%m*10000+a[i]))/m;}
    c.check();
    return c;
}
bigint bigint::operator %(bigint m){return (*this)-
((*this)/m)*m;}
bigint bigint::operator%(int m){
    if (m<0)return -((*this)%(-m));
    if (m>100000)return (*this)%bigint(m);
    int t=0;
    for (int i=length;i>=1;i--){
        t=(t*10000+a[i])%m;}
    return t;
}
void bigint::operator+=(bigint m){(*this)=(*this)+m;}
void bigint::operator-=(bigint m){(*this)=(*this)-m;}
void bigint::operator*=(bigint m){(*this)=(*this)*m;}
void bigint::operator/=(bigint m){(*this)=(*this)/m;}
void bigint::operator%=(bigint m){(*this)=(*this)%m;}
void bigint::operator*=(int m){(*this)=(*this)*m;}
void bigint::operator/=(int m){(*this)=(*this)/m;}
void bigint::operator%=(int m){(*this)=(*this)%m;}

    // 4. Input/Output
bool read(bigint &m){
    char s[maxlength*4+10];
    if (scanf("%s",&s)==-1)return false;
    for (int i=0;s[i];i++) {if (!(s[i]>='0' && s[i]<='9'
|| (s[i]=='+' || s[i]=='-') && i==0)){
            return false;}}
    m=s;
    return true;
}
void swrite(char *s,bigint m){
    int L=0;
    if (m.oper==-1) s[L++]='-';
    sprintf(s+L,"%d",m.a[m.length]);
    for (;s[L]!=0;L++);
    for (int i=m.length-1;i>=1;i--){
        sprintf(s+L,"%04d",m.a[i]);
        L+=4;
    }
    s[L]=0;
}
void write(bigint m){
    if (m.oper==-1)printf("-");
    printf("%d",m.a[m.length]);
    for (int i=m.length-1;i>=1;i--){
        printf("%04d",m.a[i]);}
}
void writeln(bigint m){
    write(m);printf("\n");
}
    // 5. Mathematical functions
bigint sqr(bigint m){return m*m;}
bigint sqrt(bigint m){
    if (m.oper<0 || m.length==0)return 0;
    bigint c,last,now,templast;
    c.length=(m.length+1)/2;
    c.a[c.length]=int(sqrt((double)m.a[c.length*2]*10000
+m.a[c.length*2-1])+1e-6);
```

```
    templast.length=c.length*2;
    templast.a[c.length*2-
1]=(c.a[c.length]*c.a[c.length])%10000;
    templast.a[c.length*2]=(c.a[c.length]*c.a[c.length])
/10000;
    templast.check();
    for (int i=c.length-1;i>=1;i--){
        last=templast;
        int head=0,tail=10000,mid,j,temp;
        while (head+1<tail){
            mid=(head+tail)/2;
            now=last;
            now.a[2*i-1]+=mid*mid;
            for (j=i+1;j<=c.length;j++){
                now.a[i+j-1]+=mid*c.a[j]*2;}
            now.length++;
            for (j=2*i-1,temp=0;j<=now.length;j++){
        now.a[j]=(temp=(temp/10000+now.a[j]))%10000;}
            now.check();
            if (now<=m){
                templast=now;
                head=mid;
            }else tail=mid;
        }
        c.a[i]=head;
    }
    c.check();
    return c;
}
bigint abs(bigint m){
    bigint c=m;
    c.oper=abs(c.oper);
    c.check();
    return c;
}
bigint gcd(bigint a,bigint b) {return
b==0?a:gcd(b,a%b);}
bigint lcm(bigint a,bigint b) {return a*(b/gcd(a,b));}
bigint max(bigint a,bigint b) {return a>b?a:b;}
bigint min(bigint a,bigint b) {return a<b?a:b;}
bigint fpow(bigint a,bigint b) {
    bigint zero = 0;
    bigint one = 1;
    bigint two = 2;
    if (b==zero) return one;
    if (b==one) return a;
    bigint half = fpow(a,b/two);
    if (b%two==0) return half*half;
    else return half*half*a;
}
```

GAUSS ELIMINATION

```
typedef vector < vector <double>> Matrix;
const double EPS = 0.000001;

Matrix M;
bool Gaussian () {
    int N = M.size();
    for( int i=0; i<N; i++ ) {
        int idx = -1;
        REP(
        for( int j = i; j < N; j++ ) {
            if (fabs (M[j][i]) > EPS) {
                idx = j;
                break;
            }
        }
        if (idx == -1){return false;}
    for( int j=i; j <= N; j++ ){swap( M[i][j], M[idx][j]
);}
    for( int j=N; j >= i; j-- ){M[i][j] /= M[i][i];}
        for( int j=i+1; j < N; j++ ) {
            for( int k=N; k >= i; k-- ) {
                M[j][k] -= M[j][i] * M[i][k];
            }
        }
    }
    for( int i=N; i >= 0; i-- ) {
        for( int j=0; j<i; j++ ) {
            for( int k=N; k >= i; k-- ) {
                M[j][k] -= M[j][i] * M[i][k];
            }
        }
    }
    return true;
}
```

NUMBER THEORY

Bertrand's Ballot Theorem

Candidate A receives p votes and candidate B receives q votes,probability that A will be strictly ahead of B throughout count?

	No Ties	Ties Allowed
	(p-q)/(p+q)	(p+1-q)/(p+1)

Gambler's Ruin Theorem

```
Fair        P_1 = n_2 / (n_1+ n_2)
            P_2 = n_1 / (n_1+ n_2)
Unfair      P_1 = (1-(p/q)^(n_2)) / (1-(p/q)^(n_1+ n_2))
            P_2 = (1-(q/p)^(n_1)) / (1-(q/p)^(n_1+ n_2))
```

Pick Theorem

Let A be the area of a simply closed lattice polygon. Let B denote the number of lattice points on the Polygon edges and I the number of points in the interior of the polygon. Then **A=I+(1/2)B-1**

Forbenius Number

If GCD is 1 then required largest amount is (X*Y)-(X+Y) and total count is (X-1)*(Y-1)/2

Erdos Gallai Theorem

A sequence of non-negative integers $d_1 \geq \cdots \geq d_n$, can be represented as the degree sequence of a finite simple graph on n vertices if and only if $d_1 + \cdots + d_n$ is even and $\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$, holds every $1 \leq k \leq n$.

Derangement

```
ll der[15];   der[0] = 1;   der[1] = 0;
REP(i,2,10) der[i] = ((ll)i-1)*(der[i-1]+der[i-2]);
```

Bell Number

The number of ways a set of n elements can be partitioned into nonempty subsets

```
ll bell[25+5][25+5];   bell[0][0] = 1;
REP(i,1,maks){
    bell[i][0] = bell[i-1][i-1];
    REP(j,1,i) bell[i][j] = bell[i-1][j-1]+bell[i][j-1];
} // Example : n=10 ==> bell[10][0]
```

Stirling Number 2nd Kind

The number of ways a set of n elements into k subsets)

```
Recurrence : S(n,k) = k*S(n-1,k) + S(n-1,k-1)
int stir[n+1][k+1];
for (int i=0; i<=n; i++) stir[i][0] = 0;
for (int i=0; i<=k; i++) stir[0][k] = 0;
REP(i,1,n) REP(j,1,i){
    if (j == 1 || i == j) stir[i][j] = 1;
    else stir[i][j] = j*stir[i-1][j] + stir[i-1][j-1];
} // Example : n=3,k=1 : stirling[3][1]
```

Carmichael Number

Composite numbers n such that a^(n-1) == 1 (mod n) for every a coprime to n.

```
// 561, 1105, 1729, 2465, 2821, 6601, 8911, 10585
// 15841, 29341, 41041, 46657, 52633, 62745, 63973
// 75361, 101101, 115921, 126217, 162401, 172081
// 188461, 252601, 278545, 294409, 314821, 334153
// 340561, 399001, 410041, 449065, 488881, 512461
```

int m,n

$$\sum_k \binom{r}{m+k}\binom{s}{n-k} = \binom{r+s}{m+n}$$

int l>=0, int m,n

$$\sum_k \binom{l}{m+k}\binom{s}{n+k} = \binom{l+s}{l-m+n}$$

int l>=0, int m,n

$$\sum_k \binom{l}{m+k}\binom{s+k}{n}(-1)^k = (-1)^{l+m}\binom{s-m}{n-l}$$

integers l,m,m>=0

$$\sum_{k\le 1} \binom{l-k}{m}\binom{s}{k-n}(-1)^k = (-1)^{l+m}\binom{s-m-1}{l-m-n}$$

integers l,m>=0,integers n>=q>=0

$$\sum_{o\le k\le l} \binom{l-k}{m}\binom{q+k}{n} = \binom{l+q+1}{m+n+1}$$

MATHEMATICS SEQUENCE FORMULA

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \qquad \sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4}$$

Triangular number sum

$$\sum_{k=0}^{n} kCm = (n+1)C(m+1)$$

S(n) = ((n)(n+1)(n+2))/6
n = (sqrt(8x + 1) -1) / 2

Faulhaber

$$\sum_{k=1}^{n} k^p = \frac{1}{p+1}\sum_{i=0}^{p}(-1)^i\binom{p+1}{i}B_i n^{p+1-i}$$

$$B_n = -\sum_{k=0}^{n-1}\binom{n}{k}\frac{B_k}{n-k+1} \quad for\ n>1; \quad 0\ for\ n=0$$

PRIME NUMBER LIST

7	99999999977
97	999999999989
997	9999999999971
9973	99999999999973
99991	999999999999989
999983	9999999999999937
9999991	99999999999999997
99999989	999999999999999989
999999937	9999999999999999961
9999999967	

Below (2^31)-1 : 2147483629 (2^31-1 : 8 Mersenne Prime)
Below (2^32)-1 : 4294967291
Below (2^63)-1 : 9223372036854775783
Below (2^64)-1 : 18446744073709551557

CATALAN & SUPER CATALAN

```
ll sc(int n) { // supercatalan
    if(n<=2) return 1;
    return (3*(2*n-3)*sc(n-1)-(n-3)*sc(n-2)) / n;
} // 1,1,1,3,11,45, 197, 903, 4279, 20793, 103049
ll c(int n) { // catalan
    if (n<=1) return 1;
    return c(n-1)*2*n*(2*n-1)/((n+1)*n);
} // 1,1,2,5,14,42, 132, 429, 1430, 4862, 16796, 58786
```

DIGIT COUNTING FACTORIAL

```
double memo[11000];
void start(){
    memo[1] = log10(1);
    REP(i,2,10000)memo[i]= memo[i-1] + (double)log10(i);
}
int solve(int n){ return (int)memo[n]+1; }
```

INT TO ROMAN

```
string toRoman(int x) {
    const string R[] = { "M", "CM", "D", "CD", "C",
"XC", "L", "XL", "X", "IX", "V", "IV", "I" };
    const int N[] = { 1000,900,500,400,100,90,50,40,
10,9,5,4,1 };
    if (x==0) return "O"; // Not decimal 0 !
    int n = sizeof(N) / sizeof(int);
    string ret="";
    FOR(i,n) while(x>=N[i]) { x-=N[i]; ret+=R[i]; }
    return ret;
}
```

nCk

```
ll res = 1;
if (n==0) { printf("%lld\n", res); continue; }
printf("%lld ", res);
for(int i=n, j=1; i>0, j<n; i--,j++) {
    res = res * i / j;
    printf("%lld ", res);
}
puts("1");
LL C(int n,int k) {
    LL numerator=1,denominator=1,toMul,toDiv,i;
    if (k>n/2) k=n-k;
    for(i=k; i; i--) {
        toMul = n-k+i;
        toDiv = i;
        GCDdiv(toMul,toDiv);
        GCDdiv(numerator,toDiv);
        GCDdiv(toMul,denominator);
        numerator *= toMul;
        denominator *= toDiv;
    }
    return numerator/denominator;
}
```

LINEAR DIOPHANTINE

```
/* given a,b,and c. This function will return true if
found a solution. Need ext_euclid().. ax + by = c!,
(d,x0,y0) = ext_euclid(a,b).. x = x0 + (b/d) * n, y =
y0 - (a/d)*n; n = 0,1,2,...
bool linear_diophantine(int a,int b,int c,int &x,int
&y) {
    int d = ext_euclid(a,b,x,y);
    if ( c%d != 0) return false;
    x *= c/d; y *= c/d;
    return true;
}
```

CRT dan kawan2

```
Assume that f(0) = 1 and 0^0=1. f(n) = (n%10)^f(n/10)
for all n bigger than zero. Please calculate f(n)%m. (2
<= n , m <= 10^9, x^y means the y th power of x).
LL inverse(LL a,LL m) {
    return mPow(a,phi(m)-1,m);
}
LL CRT( vector<pair<LL,int>> data ) {
    LL M = 1;
    FOREACH(it,data) M *= it->ND;
    LL ret = 0LL;
    FOREACH(it,data) {
        LL Mi = M / it->ND;
        LL temp = it->ST * Mi * inverse(Mi,it->ND);
        ret += temp;
        ret %= M;
    }
    return ret;
}
int nums[16], n;
const int LIM=1<<10;
LL brute(LL base,int pos, LL M) {
    if(pos==n) return base%M;
    int j = n;
    LL pj = 1;
    while(j-1 >= pos) {
        LL k = 1;
        REP(loop,pj) {
            k*=nums[j-1];
            if(k >= LIM) return 0;
        }
        pj = k; j--;
    }
    return mPow(base,pj,M);
```

```
}
LL solve(int,LL);
LL doit(int,int);
pair<LL,int> solve2(int pos, LL M) {
    LL a = nums[pos];
    if(GCD(a,M)==1) {
        LL b = solve( pos+1, phi((int)M) );
        return MP( mPow(a,b,M), M );
    }
    return MP( brute(a, pos+1, M), M);
}
LL solve(int pos,LL M) {
    if(M==1) return 0;
    if(pos==n) return 1;
    if(nums[pos]==1LL) return a%M;
    if(nums[pos]==0LL) return 0;
    LL m = M;
    vector<pair<LL,int>> data;
    for(LL i=2;i*i<=m;i++) {
        if(m%i!=0) continue;
        LL p=1;
        while(m%i==0) { m/=i; p*=i; }
        data.PB( solve2(pos, p) );
    }
    if(m>1) data.PB( solve2(pos, m) );
    return CRT(data);
}
void normalize() {
    nums[n] = 1;
    int newN = n;
    REP(i,n) {
        if(nums[i]==1) { newN = i; break;
        }else if(nums[i]==0) {
            int ctr0 = 0;
            while(nums[i+ctr0]==0) ctr0++;
            if(ctr0 % 2 == 0) nums[i] = 1, newN = i;
            else if(i-1>=0) nums[i-1]=1, newN=i-1;
            break;
        }
    }
    n=newN;   if(nums[n-1]!=1) nums[n++]=1;
}
LL doit(int x,int m) {
    n=0;
    while(x) nums[n++] = x%10, x /= 10;
    normalize();
    return solve(0,m);
}
int main() {
    int x, m;
    REP(ncase,getint()) {
        scanf("%d %d", &x, &m);
        printf("%d\n",(int)doit(x,m));
    }
    return 0;
}

//CRT Willson
int inv(int a, int m){
    int m0 = m, t, q, x0 = 0, x1 = 1;
    if(m == 1) return 0;
    while(a > 1){
        q = a / m;   t = m;   m = a % m;   a = t;
        t = x0;      x0 = x1 - q * x0;     x1 = t;
    }
    if(x1 < 0) x1 += m0; // Make x1 positive
    return x1;
}
// k is size of num[] and rem[].  Returns the smallest
// number x such that:
//    x % num[0] = rem[0]
//    x % num[1] = rem[1]
//    ..................
//    x % num[k-2] = rem[k-1]
// Assumption: Numbers in num[] are pairwise coprime
// (gcd for every pair is 1)
int findMinX(int num[], int rem[], int k){
    int prod = 1, res = 0;
    FOR(i,k) prod *= num[i]; // Compute prod of all nums
    FOR(i,k){
        int pp = prod / num[i];
        result += rem[i] * inv(pp, num[i]) * pp;
    }
    return res % prod;
}
int driver(){
    int num[] = {3, 4, 5}, rem[] = {2, 3, 1};
    int k = sizeof(num)/sizeof(num[0]);
    cout << "x is "<< findMinX(num, rem, k);
    return 0;
}
```

EULER TOTIENT (PHI) FUNCTION

```
int phi(int n) {
    if (n==1) return 0;
    int ret = n;
    for (int i=2; i*i<=n; i++) {
        if (n%i==0) ret -= ret/i;
        while(n%i==0) n/= i;
    }
    if ( n != 1) ret -= ret/n;
    return ret;
}// Sieve version
vector<int> sieve_phi(int n) {
    vector<bool> is_prime(n+1,true);
    vector<int> ret(n+1);
    FOR(i,0,n) ret[i] = i;
    FOR(i,2,n) if (is_prime[i]) {
        for(int j=i; j <= n; j += i) {
            ret[j] -= ret[j]/i;
            is_prime[j] = false;}}return ret;}
void sievephi(int n) {
    phi[1] = 0;for(LL i=2; i<n; i++) phi[i] = i;
    for(LL i=2; i<=n; (i==2LL) ? i++ : i+=2LL) {
        if ( phi[i] == i ) {
            for(LL j=i; j<=n; j+=i) {
                phi[j] = phi[j] * LL((i-1LL)) / LL(i);
            }}}
    for(LL i=2; i<=n; i++) {
        for(LL j = 1, _n = n/i; j<=_n; j++)
            ans[i*j] += j * phi[i];
    }
    for(LL i=2; i<=n; i++) ans[i] += ans[i-1];
}
int phi(int n) {
    if ( n == 1 ) return 0;
    int res = n;
    for(int i=2; i<=n/i; (i==2) ? i++ : i+=2) {
        if ( n%i ) continue;
        while(n > 1 && n%i==0) n/=i;
        if ( res % i ) res = res * (i-1) / i;
        else res = res / i * (i-1);
    }
    if ( n > 1 ) res = res / n * ( n-1 );
    return res;
}
```

PRIME TEST

```
#define MAX 100000000 #define HALF 50000000
#define SQRT 5000 #define SIZE MAX / 16 + 2
char SITO[SIZE];
#define NTH(n) ((n)&7)
#define ISPRIME(n) SITO[n>>4] & (1<< NTH(n>>1))
bool is_prime(int n) {
    if (n==2) return true;
    else if (!(n&1)) return false;
    else return ISPRIME(n);
}
void init() {
    FILL(SITO,0xFF);
```

```
    SITO[0] = 0xFE;
    for(int i=1; i<SQRT;i++)
        if (SITO[i>>3]&(1<<NTH(i)))
            for(int j=2*i*(i+1);j<HALF;j+=2*i+1)
                SITO[j>>3] &= ~(1 << NTH(j));
}
```

POLLARD RHO

```
LL pollard_rho(LL n){
    for(LL d = n,int c = 2; d==n ;c++){
        LL x = 2;   LL y = x;
        do{ // Russian peasant
            x = (mulmod(x,x,n) + c )%n;
            y = (mulmod(y,y,n) + c )%n;
            y = (mulmod(y,y,n) + c )%n;
            d = gcd(abs(x-y),n);
            if (d>1) break;
        }while(true);
    }
    pollard_rho(d),pollard_rho(n/d);
}
```

MILLER RABIN

```
bool millerTest(LL p,int iteration){
    srand(time(NULL));
    if(p<2) return false;
    if(p!=2 && p%2==0) return false;
    LL s=p-1;
    while(s%2==0) s/=2;
    for(int i=0;i<iteration;i++){
        LL a=rand()%(p-1)+1;
        LL temp=s;
        LL mod=fast_exp(a,temp,p);
        while(temp!=p-1 && mod!=1 && mod!=p-1){
            mod= mulmod(mod,mod,p); // Russian peasant
            temp *= 2;
        }
        if(mod!=p-1 && temp%2==0) return false;
    }
    return true;
}
```

EXTENDED EUCLID

```
/* given a and b for equation a*x + b*y = 1, extended
Euclid will find the x and y and returns GCD(a,b) */
int ext_euclid(int a, int b, int &x, int &y) {
    if (b==0) { x=1; y=0; return a; }
    int ret = ext_euclid(b,a%b,x,y);
    int x1 = y, y1 = x-(a/b)*y;
    x=x1, y=y1;
    return ret;
}
```

SUM OF DIVISORS

```
/* this function returns the sum of the divisors of n.
i.e div(3) will return sum of the number of divisors up
to 3. */
LL div(int n) {
    if (n==0) return 0;
    LL res = 0, i;
    for( i = 2; i*i <= n; i++) res += n/i;
    --i; res = (n*2) - (i*i) + (res*2);
    return res;
}
```

RANGE SIEVE

```
void sieve(int L,int U,vector<int>&primes) {
    memset( isnp, 0, sizeof(isnp) );
    int nto = U - L + 1;
    for(int i=0; i<primes.size(); i++) {
        int nP = primes[i];
        if ( nP > U / nP ) break;
        int fnum = (L / nP) * nP;
        if ( fnum < L ) fnum += nP;
        if ( fnum == nP ) fnum += nP;
        for(int k=fnum-L;k<nto; k += nP ) isnp[k] = 1;
    }
    if ( L <= 1 ) isnp[1-L] = 1;
    if ( L <= 0 ) isnp[0-L] = 1;}
```

MÖBIUS FUNCTION

```
int M[MAXN], SPF[MAXN];    // M=mobius, SPF=smallest pf
bool B[MAXN];              // B=is prime
vi prima;
inline void mobius() {
    prima.clear();
    memset(B,true,sizeof B);
    B[0]=B[1]=false;
    REP(i,2,MAXN-1){
        if(B[i])prima.pb(i),SPF[i]=i;
        FOR(auto j:prima){
            if(!(i*j<MAXN && j<=SPF[i]))break;
            B[i*j]=false; SPF[i*j]=j;
        }
    }
    REP(i,1,MAXN-1){
        if (i==1) M[i]=1;
        else if (SPF[i]==i) M[i]=-1;
        else if ((i/SPF[i])%SPF[i]==0) M[i]=0;
        else M[i]=M[i/SPF[i]]*M[SPF[i]];
    }
}
```

GEOMETRIC TRANSFORMATION

Cermin: $\begin{bmatrix}\cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta\end{bmatrix}$ Rotasi: $\begin{bmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{bmatrix}$

Dilatasi: $\begin{bmatrix}k & 0 \\ 0 & k\end{bmatrix}$ Gusuran/Sharing: $\begin{bmatrix}1 & x \\ y & 1\end{bmatrix}$

Regangan: $\begin{bmatrix}x & 0 \\ 0 & y\end{bmatrix}$

TRIGONOMETRIC EQUATIONS

Sin(3A) = -4 Sin³ A + 3 Sin A
Cos(3A) = 4 Cos³ A – 3 Cos A
Sin(A/2) = ((1 – Cos A)/2)^0.5
Cos(A/2) = ((1 + Cos A)/2)^0.5
Tan A + Tan B + Tan C = Tan A Tan B Tan C ; A+B+C= PI

2 Sin A Cos B = Sin(A+B) + Sin(A-B)
2 Cos A Sin B = Sin(A+B) – Sin(A-B)
2 Cos A Cos B = Cos(A+B) + Cos(A-B)
-2 Sin A Sin B = Cos(A+B) – Cos(A-B)

Rumus Sin Cos Segitiga

a/Sin A = b/Sin B = c/Sin C
a² = b² + c² - 2 bc Cos A
L = (ab Sin C)/2 = (bc Sin A)/2 = (ac Sin B)/2
L = (c^2 * sin A * sin B) / (2 * sin C)

Heron's Formula

L = (s(s-a)(s-b)(s-c))^0.5
s = (a+b+c)/2 r = L/ s R = abc / 4L

Luas n-gon
L = (n r r Sin(360/n))/2

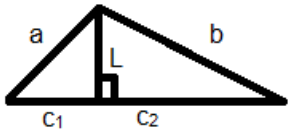
Lingkaran Singgung Segitiga sisi:
a: (s(s-b)(s-c)/(s-a))^0.5
b: (s(s-a)(s-c)/(s-b))^0.5
c: (s(s-a)(s-b)/(s-c))^0.5

$$m = \sqrt{\frac{2b^2 + 2c^2 - a^2}{4}}$$

Titik ke Garis

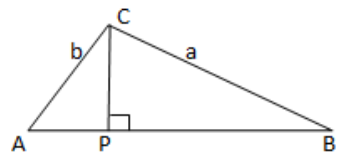
U = ((x3-x1)(x2-x1)+(y3-y1)(y2-y1))/((x2-x1)²+(y2-y1)²)
X = x1 + U(x2-x1)
Y = y1 + U(y2-y1)

Dalil Steward



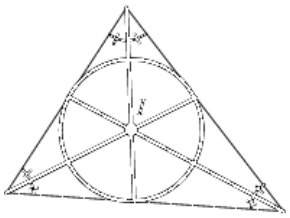
$L^2 \cdot c = a^2.c_2 + b^2.c_1 - c_1.c_2.c$

Dalil Proyeksi

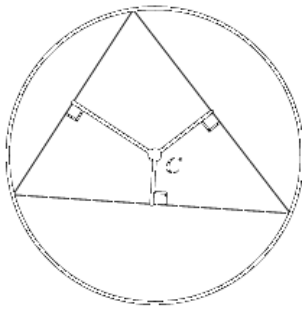


c

AP = (b^2 + c^2 - a^2) / 2c



incircle



circumcircle

Law of Tangents

$$\frac{a-b}{a+b} = \frac{\tan^1/_2(\alpha-\beta)}{\tan^1/_2(\alpha+\beta)}$$

$$\frac{b-c}{b+c} = \frac{\tan^1/_2(\beta-\gamma)}{\tan^1/_2(\beta+\gamma)}$$

Mollweide's Formula

$$\frac{a+b}{c} = \frac{\cos^1/_2(\alpha-\beta)}{\sin^1/_2\gamma}$$

$$\frac{a-c}{a+c} = \frac{\tan^1/_2(\alpha-\gamma)}{\tan^1/_2(\alpha+\gamma)}$$

POLYGON CENTROID AND AREA

$$C_x = \frac{1}{6A}\sum_{i=0}^{n-1}(x_i + x_{i+1})(x_i\ y_{i+1} - x_{i+1}\ y_i)$$

$$C_y = \frac{1}{6A}\sum_{i=0}^{n-1}(y_i + y_{i+1})(x_i\ y_{i+1} - x_{i+1}\ y_i)$$

$$A = \frac{1}{2}\sum_{i=0}^{n-1}(x_i\ y_{i+1} - x_{i+1}\ y_i)$$

(x(n),y(n)) == (x0,y0)

LINE SEGMENT INTERSECTION POINT

$$Y = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & \begin{vmatrix}y_1 & 1\\ y_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & \begin{vmatrix}y_3 & 1\\ y_4 & 1\end{vmatrix}\end{vmatrix}}{\begin{vmatrix}\begin{vmatrix}x_1 & 1\\ x_2 & 1\end{vmatrix} & \begin{vmatrix}y_1 & 1\\ y_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & 1\\ x_4 & 1\end{vmatrix} & \begin{vmatrix}y_3 & 1\\ y_4 & 1\end{vmatrix}\end{vmatrix}} = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & y_1 - y_2\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & y_3 - y_4\end{vmatrix}}{\begin{vmatrix}x_1 - x_2 & y_1 - y_2\\ x_3 - x_4 & y_3 - y_4\end{vmatrix}},$$

$$X = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & \begin{vmatrix}x_1 & 1\\ x_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & \begin{vmatrix}x_3 & 1\\ x_4 & 1\end{vmatrix}\end{vmatrix}}{\begin{vmatrix}\begin{vmatrix}x_1 & 1\\ x_2 & 1\end{vmatrix} & \begin{vmatrix}y_1 & 1\\ y_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & 1\\ x_4 & 1\end{vmatrix} & \begin{vmatrix}y_3 & 1\\ y_4 & 1\end{vmatrix}\end{vmatrix}} = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & x_1 - x_2\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & x_3 - x_4\end{vmatrix}}{\begin{vmatrix}x_1 - x_2 & y_1 - y_2\\ x_3 - x_4 & y_3 - y_4\end{vmatrix}}$$

LINE SEGMENT INTERSECTION

```
struct Vertex { int x, y; };
int GetArea2(Vertex& a, Vertex& b, Vertex& c) {
    return (b.x-a.x)*(c.y-a.y)-(c.x-a.x)*(b.y-a.y);
}
bool OnSegment(Vertex& a, Vertex& b, Vertex& c) {
    return(min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x)
&& min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y));
}
bool IsIntersect(Vertex& a, Vertex& b, Vertex& c,
Vertex& d) {
    int d1, d2, d3, d4;
    d1 = GetArea2(c, d, a);
    d2 = GetArea2(c, d, b);
    d3 = GetArea2(a, b, c);
    d4 = GetArea2(a, b, d);

    if(((d1 > 0 && d2 < 0) ||(d1 < 0 && d2 > 0)) &&(( d3
> 0 && d4 < 0) || d3 < 0 && d4 > 0)) { return true; }
    if(d1 == 0 && OnSegment(c, d, a)) {return true;}
    if(d2 == 0 && OnSegment(c, d, b)) {return true;}
    if(d3 == 0 && OnSegment(a, b, c)) {return true;}
    if(d4 == 0 && OnSegment(a, b, d)) {return true;}
    return false;
}
bool IsInside(Vertex& a, Vertex& lt, Vertex& rb) {
    return(min(lt.x, rb.x) <= a.x && a.x <= max(lt.x,
rb.x) && min(lt.y, rb.y) <= a.y && a.y <= max(lt.y,
rb.y));
}
// line intersection
struct Vertex {
    int x, y; Vertex result;
    Vertex operator - (Vertex& b) {
        result.x = x - b.x;
        result.y = y - b.y;
        return result;
    }
};
void GetIntersectionAttribute (Vertex& a, Vertex& b,
Vertex& c, Vertex& d, int& fN, int& sN, int& D) {
    fN = (d.x-c.x) * (a.y-c.y) - (d.y-c.y) * (a.x-c.x);
    sN = (b.x-a.x) * (a.y-c.y) - (b.y-a.y) * (a.x-c.x);
    D = (d.y-c.y) * (b.x-a.x) - (d.x-c.x) * (b.y-a.y);
}
double GetIntersectionTime (int N, int D) {
    return (double) N / (double) D;
}
double GetIntersectionX (Vertex& a, Vertex& b, double
t) { return a.x + (b - a).x * t; }
double GetIntersectionY (Vertex& a, Vertex& b, double
t) { return a.y + (b - a).y * t; }
```

SQUARE COLLISION

```
((x1>=x1t && x1<=x2t) && ((y1>=y1t && y1<=y2t) ||
(y2>=y1t && y2<=y2t) || (y1<=y1t && y2>=y2t))) ||
((x2>=x1t && x2<=x2t) && ((y1>=y1t && y1<=y2t) ||
(y2>=y1t && y2<=y2t) || (y1<=y1t && y2>=y2t))) ||
((x1<=x1t && x2>=x2t) && ((y1>=y1t && y1<=y2t) ||
(y2>=y1t && y2<=y2t) || (y1<=y1t && y2>=y2t)))
```

CLOSEST PAIR

```
const double inf = 10000.0;
const double eps = 1e-9;
typedef long long ll;
struct point {double x, y;};
int cmp(const point &a, const point &b) {
    return a.x < b.x;
}
inline double edist(point &a, point &b) {
    return sqrt((a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-
b.y));
}
inline double dabs(double a) {
    return (a < 0) ? -a : a;
}
inline double dmin(double a, double b) {
    return (a < b) ? a : b;
```

```
}
int n, buff[10005], Y[10005], Yak[10005];
point X[10005];
inline double conquer(int l,int mid,int r,double dn) {
    for (int i=l,j=mid+1,k=l; i<=mid || j<=r; k++) {
        if ((j > r) || (i<=mid && X[Y[i]].y <X[Y[j]].y))
            buff[k] = Y[i], i++;
        else buff[k] = Y[j], j++;
    }
    int nak = 0;
    for (int i = l; i<=r; i++) {
        Y[i] = buff[i];
        if (dabs(X[mid].x - X[Y[i]].x) <= dn) {
            Yak[nak++] = Y[i];
        }
    }
    double ret = dn;
    for (int i = 0; i < nak; i++) {
        for (int j = i+1; j < nak; j++) {
            if (dabs(X[Yak[i]].y-X[Yak[j]].y)>ret)break;
            ret = dmin(ret,edist(X[Yak[i]], X[Yak[j]]));
        }
    }
    return ret;
}
inline double divide(int l, int r) {
    double ret = inf;
    if (l < r) {
        double dl = divide(l, (l+r)/2);
        double dr = divide((l+r)/2+1, r);
        ret = conquer(l, (l+r)/2, r, dmin(dl, dr));
    }
    return ret;
}
int main() {
    while (scanf("%d", &n) != EOF && n) {
        rep(i, n) scanf("%lf %lf", &X[i].x, &X[i].y);
        sort(X, X+n, cmp);
        rep(i, n) Y[i] = i;
        double ans = divide(0, n-1);
        if (dabs(ans-inf) <= eps) printf("INFINITY\n");
        else printf("%.4lf\n", ans);
    }
}
```

POLYGON CLASS

```
class Line {
    public :
    Point p1, p2;
    Line(){}
    Line(Point _p1, Point _p2):p1(_p1),p2(_p2){}
    bool intersect(double x1, double y1, double x2,
double y2, double x3, double y3, double x4, double
y4,double &x, double &y) {
        double nom = (x2-x1)*(y1-y3) + (y2-y1)*(x3-x1);
        double den = (x2-x1)*(y4-y3)-(x4-x3)*(y2-y1);
        if (den==0) return false;
        double b = nom/den;
        x = x3 + (x4-x3) * b;
        y = y3 + (y4-y3) * b;
        double a = (x2-x1 == 0) ? ((y-y1) / (y2-y1)) :
((x-x1) / (x2-x1));
        return 0<=a && a<=1 && 0<=b && b<=1;
    }
    bool intersect(Line &that, Point &p) {
        return intersect(p1.x, p1.y, p2.x, p2.y,
that.p1.x, that.p1.y, that.p2.x, that.p2.y, p.x, p.y);
    }
};
class Polygon {
    public :
    VP P;
    Polygon(){}
    Polygon(VP _P):P(_P){}
    void addPoint(Point p){ P.push_back(p); }
    Polygon intersect(Polygon &that) {
        double qarea = that.area(), parea = area();
        VP &Q = that.P;
        set<Point> p;
        Point pt;
        REP(i,P.size()){
            int ni = (i+1)%P.size();
            Point p1(P[i].x, P[i].y);
            Point p2(P[ni].x,P[ni].y);
            Line L1(p1, p2);
            REP(j,Q.size()){
                int nj = (j+1)%Q.size();
                Point p3(Q[j].x, Q[j].y);
                Point p4(Q[nj].x,Q[nj].y);
                Line L2(p3, p4);
                if (L1.intersect(L2,pt)) p.insert(pt);
            }
        }
        REP(i,P.size()){
            double a = 0;
            REP(j,Q.size()) a += fabs(det(Q[j],
            Q[(j+1)%Q.size()], P[i])/2);
            if (fabs(a-qarea) < EPS) p.insert(P[i]);
        }
        REP(i,Q.size()){
            double a = 0;
            REP(j,P.size()) a += fabs(det(P[j],
            P[(j+1)%P.size()], Q[i])/2);
            if (fabs(a-parea) < EPS) p.insert(Q[i]);
        }
        return Polygon(cHull(VP(p.begin(),p.end())));
    }
    double area(){
        double ret = 0;
        REP(i, P.size()){
            int ni = (i+1)%P.size();
            ret += P[i].x * P[ni].y - P[i].y * P[ni].x;
        }
        return fabs(ret/2);
    }
};
int main(){
    for (int nA,nB,x,y; scanf("%d",&nA)!=EOF && nA; ){
        Polygon P, Q;
        REP(i,nA) {
            scanf("%d %d",&x,&y);
            P.addPoint(Point(x,y));
        }
        scanf("%d",&nB);
        REP(i,nB) {
            scanf("%d %d",&x,&y);
            Q.addPoint(Point(x,y));
        }
        Polygon I = P.intersect(Q);
        printf("%8.2lf\n”,P.area()+Q.area()-2*I.area());
    }
    puts(“”);
    return 0;
}
```

PT TO LINE & LINE TO SEGMENT DIST

```
// Point to line
Returns the distance from p to the line defined by {a,
b}.The closest point on the line is returned through
(cpx,cpy). Does not work for degenerate lines (when
answer isundefined).
REQUIRES: dist()
double distToLine(
    double ax, ay, bx, by, px, py;
    double *cpx, double *cpy ) {
// Formula: cp = a + (p-a).(b-a) / |b-a| * (b-a)
double proj = ((px - ax)*(bx - ax) + (py - ay)*(by -
ay)) /((bx - ax) * (bx - ax) + (by - ay) * (by - ay));
        *cpx = ax + proj * ( bx - ax );
        *cpy = ay + proj * ( by - ay );
```

```
        return dist(px, py, *cpx, *cpy);
}
// Line to segment
Returns the distance from p to the line segment ab.
Theclosest point on ab is returned through (cpx, cpy).
Workscorrectly for degenerate line segments (a == b).
REQUIRES: dist(), distToLine()
double distToLineSegment(
    double ax, double ay,
    double bx, double by,
    double px, double py,
    double *cpx, double *cpy) {
    if ((bx - ax)*(px - ax)+(by - ay)*(py - ay)<EPS ) {
        *cpx = ax; *cpy = ay;
        return dist(ax, ay, px, py);
    }
    if ((ax - bx)*(px - bx)+(ay - by)*(py - by)<EPS ) {
        *cpx = bx; *cpy = by;
        return dist(bx, by, px, py);
    }
    return distToLine(ax, ay, bx, by, px, py, cpx, cpy);
}
```

POLYGON DIAMETER

```
struct ConvexPolygon {
    Point *p;
    int n;
    ConvexPolygon(Point *pp, int nn) {p = pp; n = nn;}
double diameter() {
    int a, b;
    double d;
    double area1, area2;
    d = 0;
    if(n == 2) return getDist(p[0], p[1]);
    if(n == 3) return max(getDist(p[0],p[1]),
getDist(p[1], p[2]), getDist(p[0], p[2]));
    a = n - 1; b = 1;
    area1 = area(p[a], p[(a + 1) % n], p[(b + 1) % n]);
    area2 = area(p[a], p[(a + 1) % n], p[b]);
    while(area1 > area2) {
        b = (b + 1) % n;
        area1 = area(p[a], p[(a + 1) % n], p[(b+1)%n]);
        area2 = area(p[a], p[(a + 1) % n], p[b]);
    }
    while(b != 0) {
        a = (a + 1) % n;
        d = max(d, distance2Point(p[a], p[b]));
        area1 = area(p[a], p[(a + 1) % n], p[(b+1)%n]);
        area2 = area(p[a], p[(a + 1) % n], p[b]);
        while(area1 > area2) {
            b = (b + 1) % n;
            area1 = area(p[a], p[(a+1)%n], p[(b+1)%n]);
            area2 = area(p[a], p[(a + 1) % n], p[b]);
            if(b == 0) return d;
            d = max(d, distance2Point(p[a], p[b]));
        }
    }
    return d;
}};
```

UNION RECTANGLE INTERSECTION

```
struct event{ int ind, type; };
struct point{ double x,y; };

event events_v[350],events_h[350];
point rect[150][3];
bool in_set[350];
int n;

bool compare_x(event a,event b){
    return rect[a.ind][a.type].x<rect[b.ind][b.type].x;
}
bool compare_y(event a,event b){
    return rect[a.ind][a.type].y<rect[b.ind][b.type].y;
}
double union_area(int e){
    memset(in_set,false,sizeof(in_set));
    sort(events_v,events_v + e, compare_x);
    sort(events_h,events_h + e, compare_y);
    double area = 0;
    in_set[events_v[0].ind] = 1;
    for(int i=1;i<e;i++){
        event c = events_v[i];
        int cnt = 0;
        double delta_x = rect[c.ind][c.type].x -
rect[events_v[i-1].ind][events_v[i-1].type].x;
        double begin_y;
        if(delta_x == 0){
            in_set[c.ind] = (c.type == 0);
            continue;
        }
        for(int j=0;j<e;j++){
            if(in_set[events_h[j].ind] == 1){
                if(events_h[j].type == 0){
                    if(cnt == 0) begin_y =
rect[events_h[j].ind][0].y;
                    ++cnt;
                }
                else if(--cnt == 0){
                    double delta_y =
rect[events_h[j].ind][1].y - begin_y;
                    area += delta_x * delta_y;
                }
            }
        }
        in_set[c.ind ] = (c.type == 0);
    }
    return area;
}
int main(){
    int tc = 0;
    while(cin>>n && n!=0){
        tc++;
        for(int a=0;a<n;a++){
            double x,y,r;
            cin>>x>>y>>r;
            rect[a][0].x = x-r;
            rect[a][0].y = y-r;
            rect[a][1].x = x+r;
            rect[a][1].y = y+r;
            events_v[2*a].ind = a;
            events_v[2*a].type = 0;
            events_v[2*a + 1].ind = a;
            events_v[2*a + 1].type = 1;
            events_h[2*a].ind = a;
            events_h[2*a].type = 0;
            events_h[2*a + 1].ind = a;
            events_h[2*a + 1].type = 1;
        }
        double area = union_area(2*n);
        printf("%d %.2lf\n",tc,area);
    }
}
```

GREAT CIRCLE DISTANCE

```
const double R = 6371.01;
double a=sin(plat)*sin(qlat);
double b=cos(plat)*cos(qlat)*cos(plong)*cos(qlong);
double c=cos(plat)*cos(qlat)*sin(plong)*sin(qlong);
return R * acos(a + b + c);
```

END OF TEAM NOTEBOOK